

Interview Cracker

Complete interview question bank

Generated from the Markdown sources in the Interview Cracker repository. Every code snippet in these pages was executed and checked before publication.

Data Structures

Arrays and Strings

The most common interview substrate. Most "easy" and "medium" problems are an array or a string with a pattern applied to it.

[Easy] Q. What is the difference between an array and a dynamic array?

Answer. A **static array** is a fixed-size, contiguous block of memory. Its size is decided at creation and never changes. `int[10]` in C or Java.

A **dynamic array** wraps a static array and grows it when it fills up, by allocating a larger block and copying. Python `list`, Java `ArrayList`, C++ `vector`, Go `slice`, JavaScript `Array`.

Both give $O(1)$ indexed access, because the address of element `i` is just `base + i * element_size - one` multiplication and one addition, no traversal.

Operation	Cost	Why
Access by index	$O(1)$	Direct address arithmetic
Append	$O(1)$ amortised	Occasional doubling + copy
Insert/delete at front or middle	$O(n)$	Everything after must shift
Search (unsorted)	$O(n)$	No structure to exploit
Search (sorted)	$O(\log n)$	Binary search

[Easy] Q. Why is inserting at the front of an array $O(n)$ but appending $O(1)$?

Answer. Because arrays are contiguous. Inserting at index 0 means every existing element must move one slot to the right to make room - n moves. Appending writes into already-reserved space at the end - one move.

This is the single most useful performance fact about arrays, and it is why:

- `list.pop(0)` is a performance bug in a loop; use `collections.deque`.
 - Building a result by `insert(0, x)` is $O(n^2)$; append and reverse at the end.
-

[Medium] Q. Why are strings immutable in Python, Java and JavaScript?

Answer. Several reasons reinforce each other:

- **Safe sharing.** An immutable string can be passed around, cached and shared
- **Hashability.** A string's hash can be computed once and cached. If strings
- **Interning.** Identical literals can share one object, saving memory.
- **Security.** A filename or URL validated and then passed to the OS cannot be

The cost is that every "modification" allocates. Concatenating in a loop is $O(n^2)$; use a builder (`StringBuilder`, `"".join`, `array + join`) instead.

[Medium] Q. What does "in-place" mean, and what is the space complexity of an in-place algorithm?

Answer. In-place means the algorithm transforms the input using only $O(1)$ extra space - a few variables, no second array proportional to n .

```
def reverse_in_place(arr):          # O(1) extra space
    left, right = 0, len(arr) - 1
    while left < right:
        arr[left], arr[right] = arr[right], arr[left]
        left += 1
        right -= 1
```

Two honest caveats interviewers probe for:

- You cannot do this to a string in Python/Java - strings are immutable, so you
- $O(\log n)$ stack space from recursion is usually still called "in-place" by

[Medium] Q. What is the two-pointer technique and when does it apply?

Answer. Two indices moving through the array under a rule, replacing a nested loop - turning $O(n^2)$ into $O(n)$.

Three shapes:

Converging (from both ends) - needs a **sorted** array or a symmetry:

```
left, right = 0, len(arr) - 1
while left < right:
    total = arr[left] + arr[right]
    if total == target: return (left, right)
    if total < target: left += 1      # need bigger
    else:              right -= 1    # need smaller
```

This works only because sorting guarantees which direction increases the sum.

Fast/slow - cycle detection, finding the middle of a linked list.

Same direction (read/write) - in-place filtering, deduplication:

```
write = 0
for read in range(len(arr)):
    if keep(arr[read]):
        arr[write] = arr[read]
        write += 1
return write          # new logical length
```

The tell: the array is sorted, or you are looking for a pair/triplet, or you must filter in place.

[Medium] Q. What is the sliding window technique?

Answer. A window `[left, right]` over a contiguous run, where `right` expands and `left` contracts to restore a constraint. Each element enters and leaves at most once, so the whole scan is $O(n)$ despite the nested-looking loops.

```
def longest_unique(s):
    last_seen = {}
    left = best = 0
    for right, ch in enumerate(s):
        if ch in last_seen and last_seen[ch] >= left:
            left = last_seen[ch] + 1      # contract past the duplicate
        last_seen[ch] = right
        best = max(best, right - left + 1)
    return best
```

The tell: "longest/shortest/maximum **contiguous** subarray or substring satisfying X". If the problem says subsequence (non-contiguous), a window does not apply - that is usually dynamic programming.

[Medium] Q. How do you find a duplicate in an array? Give three approaches.

Answer. The interviewer wants to see you trade time against space.

- 1 **Brute force** - compare every pair. $O(n^2)$ time, $O(1)$ space.
- 2 **Sort first** - duplicates become adjacent. $O(n \log n)$ time, $O(1)$ extra space
- 3 **Hash set** - one pass, check membership. $O(n)$ time, $O(n)$ space.

Then the constrained follow-up: values are in $1 \dots n$ and you may not modify the input or use extra space. That is Floyd's cycle detection applied to the array as an implicit linked list ($i \rightarrow \text{arr}[i]$), giving $O(n)$ time and $O(1)$ space.

Say all three and name the trade-off explicitly - that is the answer they want, not just the fastest one.

[Easy] Q. What is the difference between a subarray, a subsequence and a subset?

Answer. A distinction that changes which algorithm applies, so get it right:

Term	Contiguous?	Order kept?	Count for n elements
Subarray / substring	Yes	Yes	$n(n+1)/2$
Subsequence	No	Yes	2^n
Subset	No	No	2^n

For $[1, 2, 3]$: $[2, 3]$ is a subarray; $[1, 3]$ is a subsequence but not a subarray; $\{3, 1\}$ is a subset.

Why it matters: subarray problems usually yield to a sliding window or prefix sums ($O(n)$). Subsequence problems usually need dynamic programming, because there are exponentially many.

[Medium] Q. What is a prefix sum and what is it for?

Answer. A precomputed array where `prefix[i]` is the sum of the first i elements. It makes any range sum $O(1)$:

```
prefix = [0]
for n in nums:
    prefix.append(prefix[-1] + n)

# sum of nums[i:j] in O(1)
range_sum = prefix[j] - prefix[i]
```

Build cost $O(n)$ once, then unlimited $O(1)$ range queries - the right trade when you answer many queries over a static array. Combined with a hash table of seen prefix sums it solves "count subarrays summing to k" in one pass.

Worked examples: [coding-challenges/two-pointers/](#) and [coding-challenges/sliding-window/](#).

Hash Tables

The most useful data structure in interviews. A very large fraction of "optimise this $O(n^2)$ solution" problems are solved by adding a hash table.

[Easy] Q. How does a hash table work?

Answer. It stores key-value pairs in an array, and uses a hash function to compute which slot a key belongs in.

- 1 `hash(key)` produces an integer.
- 2 That integer is reduced to an index, usually `hash % capacity`.
- 3 The pair is stored at that index.

Lookup repeats the same computation, so it goes straight to the slot instead of scanning - $O(1)$ average instead of $O(n)$.

The whole structure rests on one assumption: that the hash function spreads keys roughly evenly. Everything that goes wrong with hash tables is that assumption failing.

[Easy] Q. What is a collision, and how are collisions resolved?

Answer. A collision is two different keys mapping to the same slot. They are unavoidable - there are more possible keys than slots (the pigeonhole principle).

Two families of solution:

Separate chaining. Each slot holds a list of entries. On collision, append. Lookup scans that one short list. Simple, degrades gracefully, but costs a pointer per entry and scatters memory. Used by Java's `HashMap` (which upgrades a long chain to a balanced tree) and most C++ `unordered_map` implementations.

Open addressing. Everything lives in the array itself. On collision, probe for another free slot - linearly, quadratically, or via a second hash. Better cache behaviour, no per-entry allocation, but deletion is awkward (you must leave a tombstone, or later probes stop too early) and it degrades badly when full. Used by Python's `dict` and Go's maps.

[Medium] Q. Hash table lookup is " $O(1)$ ". When is it not?

Answer. $O(1)$ is the average case. The worst case is $O(n)$, when every key hashes to the same slot and the table degenerates into a linked list.

This matters in practice in two ways:

- **Bad hash functions.** If you write `hashCode()` as `return 1`, every lookup
- **Hash-flooding attacks.** If an attacker can choose your keys and knows your

Java 8+ mitigates it differently: a bucket whose chain exceeds 8 entries converts to a red-black tree, capping the worst case at $O(\log n)$.

[Medium] Q. What is the load factor and why does it trigger a resize?

Answer. Load factor = entries / capacity. It measures how full the table is.

As it rises, collisions rise, and average probe length rises with it. Past roughly 0.7 the performance falls off sharply, so implementations resize - double the capacity and **rehash every entry**, because the index depends on capacity.

That rehash is $O(n)$, which makes a single insert occasionally $O(n)$ - but amortised $O(1)$ across all inserts, for exactly the same geometric-growth reason a dynamic array append is.

Practical consequence: if you know you will insert a million entries, pre-size the map. You avoid about 20 rehashes.

[Medium] Q. What makes a good hash function?

Answer. Three properties:

- 1 **Deterministic** - the same key always hashes to the same value. Within one
- 2 **Uniform** - outputs spread evenly across the range, so no slot is favoured.
- 3 **Fast** - it runs on every single operation.

Note that cryptographic strength is not required and usually not wanted; SHA-256 is uniform but far too slow for a hash table.

[Hard] Q. What is the contract between equals and hashCode?

Answer. If two objects are equal, they must have the same hash code. The converse is not required - unequal objects may share a hash code, that is just a collision.

Break it and hash tables break silently:

```
Set<Point> set = new HashSet<>();
set.add(new Point(1, 2));
set.contains(new Point(1, 2)); // false, if hashCode() was not overridden
```

The lookup hashes to a different slot and never finds the entry, even though `equals` would have said yes.

The other half of the contract: **a key's hash must not change while it is in the table**. Mutating a field that feeds `hashCode` strands the entry in the wrong slot - it is in the table but unreachable, and it will not even be removed by `remove()`. This is why immutable keys are strongly preferred, and why Python simply refuses to hash a `list`.

[Medium] Q. When would you not use a hash table?

Answer. Four cases:

- **You need ordering.** Hash tables have none. Use a balanced BST (`TreeMap`,
 - **You need the worst case bounded.** Real-time systems cannot accept an
 - **The dataset is tiny.** For five entries, a linear scan of an array is faster
 - **Keys are not hashable.** Mutable or unorderable keys.
-

[Medium] Q. What is the difference between a HashMap, a HashSet and a HashTable?

Answer.

- **Map** - key -> value. The core structure.
 - **Set** - keys only. Almost always implemented as a map with a dummy value.
 - **Hashtable** (the legacy Java class, capital-T) - a synchronised, slower
-

[Medium] Q. How do you use a hash table to turn $O(n^2)$ into $O(n)$?

Answer. The pattern: **replace an inner search loop with a lookup**, trading memory for time.

Two Sum is the canonical example. The brute force checks every pair:

```
for i in range(len(nums)):          #  $O(n^2)$ 
    for j in range(i + 1, len(nums)):
        if nums[i] + nums[j] == target:
            return [i, j]
```

The hash table version asks, for each number, "have I already seen the value that completes this pair?" - a lookup instead of a scan:

```
seen = {}                          # value -> index
for i, n in enumerate(nums):       #  $O(n)$  time,  $O(n)$  space
    if target - n in seen:
        return [seen[target - n], i]
    seen[n] = i
```

Recognising this - "I am searching inside a loop, so I should index instead" - is worth more than memorising any individual solution. Worked examples are in [coding-challenges/hashing/](#).

Linked Lists

Rarely the right choice in production, reliably popular in interviews - because they test pointer manipulation and careful edge-case handling with no library to hide behind.

[Easy] Q. What is a linked list and how does it differ from an array?

Answer. A chain of nodes, each holding a value and a reference to the next node. Nodes live anywhere in memory; the links impose the order.

	Array	Linked list
Access by index	O(1)	O(n) - must walk
Insert/delete at head	O(n)	O(1)
Insert/delete at a known node	O(n)	O(1)
Memory per element	value only	value + pointer(s)
Cache locality	Excellent	Poor
Size	grows by reallocation	grows one node at a time

The honest summary: linked lists win only when you are inserting or deleting at a position you already hold a reference to, and never index. In practice, cache locality means an array often wins even then - a linear scan of a contiguous array can outrun pointer-chasing by an order of magnitude.

[Easy] Q. Singly vs doubly linked - what does the extra pointer buy?

Answer. A `prev` pointer costs one word per node and buys:

- **Backwards traversal.**
- **O(1) deletion given only the node** - a singly linked list needs the
- **O(1) append with a tail pointer** - also possible singly, but removing the

This is why an LRU cache uses a doubly linked list: it must move an arbitrary node to the front in O(1), which requires unlinking it, which requires `prev`.

[Medium] Q. How do you detect a cycle in a linked list?

Answer. Floyd's tortoise and hare. Two pointers, one moving one step per iteration and one moving two. If there is a cycle they must eventually meet; if there is not, the fast pointer reaches the end.

```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    if slow is fast:
        return True
    return False
```

O(n) time, O(1) space. The hash-set alternative (store every visited node) is also O(n) time but O(n) space - mention both, then say why you would pick Floyd.

Why they must meet: once both are inside the cycle, the fast pointer gains exactly one position per iteration, so it closes any gap of size k in k steps. It cannot "jump over" the slow pointer.

The follow-up - find where the cycle starts. After they meet, reset one pointer to the head and advance both one step at a time. They meet at the cycle entrance. (The distance from head to entrance equals the distance from the meeting point to the entrance, going forward around the loop.)

[Medium] Q. Reverse a singly linked list.

Answer. Three pointers, walking once and flipping each link as you pass.

```
def reverse(head):
    prev = None
    curr = head
    while curr:
        next_node = curr.next    # save it BEFORE overwriting
        curr.next = prev        # flip
        prev = curr              # advance
        curr = next_node
    return prev                  # new head
```

O(n) time, O(1) space. The whole problem is saving `curr.next` before you destroy it - do that in the wrong order and you lose the rest of the list.

The recursive version is elegant but **O(n) space** from the call stack, which is a real downside worth naming rather than presenting it as equally good.

[Medium] Q. Find the middle of a linked list in one pass.

Answer. Same fast/slow trick. When the fast pointer reaches the end, the slow pointer is at the middle.

```
def middle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow
```

For even lengths this returns the second of the two middles. If you need the first, stop when `fast.next` and `fast.next.next`. Interviewers often care which one you return - say out loud which convention you chose.

[Medium] Q. Why does so much linked-list code use a dummy head node?

Answer. Because it removes the "what if the head changes?" special case.

Without a dummy, deleting the first node or inserting before it needs separate branching, and that branch is where the bug always is. With a sentinel node in front, every node has a predecessor and one uniform code path handles all of them:

```
def remove_all(head, target):
    dummy = Node(None)
    dummy.next = head
    prev = dummy
    while prev.next:
        if prev.next.value == target:
            prev.next = prev.next.next    # works even for the real head
        else:
            prev = prev.next
    return dummy.next                    # not `head` - it may be gone
```

Reaching for a dummy node unprompted signals real experience.

[Hard] Q. Merge two sorted linked lists. What is the complexity, and why is it nicer than for arrays?

Answer. Walk both with a pointer each, always taking the smaller head, and splice it onto a result list built behind a dummy node.

$O(n + m)$ time, $O(1)$ extra space - and that $O(1)$ is the interesting part. The array version of merge needs an $O(n + m)$ output buffer, because you cannot make room in a contiguous block without copying. A linked list merge just re-points existing nodes, allocating nothing.

This is the one place linked lists genuinely beat arrays, and it is why merge sort on a linked list is $O(1)$ -space while merge sort on an array is $O(n)$ -space.

A worked implementation is in `examples/java/algorithms/merge_two_sorted_lists.java` and `examples/c/data-structures/linked_list.c`.

[Medium] Q. What edge cases must linked-list code handle?

Answer. Interviewers grade heavily on these, and they are the same four every time:

- 1 **Empty list** (`head` is `None`).
- 2 **Single node** (`head.next` is `None`) - many loops never execute.
- 3 **Operating on the head** - does your code return the right new head?
- 4 **Operating on the tail** - is `next` set to `None`, or did you leave a

Walk through a 0-node, 1-node and 2-node list out loud before you say you are done. It takes twenty seconds and catches most bugs.

Stacks, Queues and Heaps

Three small structures that unlock a surprising number of problems once you learn to recognise their signals.

[Easy] Q. Stack vs queue?

Answer. Both restrict where you may add and remove; they differ in which end.

- **Stack** - LIFO, last in first out. `push/pop` at the same end.
- **Queue** - FIFO, first in first out. Add at the back, remove from the front.

All four operations are $O(1)$ in a good implementation.

Where each shows up:

Stack	Queue
Function call stack	BFS / level-order traversal
Undo history	Task and message queues
Bracket matching	Print spoolers, request buffers
DFS (explicit or via recursion)	Rate limiting, buffering
Expression evaluation	Producer-consumer pipelines

The rule of thumb: DFS uses a stack, BFS uses a queue. That one fact decides a lot of problems.

[Easy] Q. How do you implement a queue efficiently?

Answer. Not with a plain array and `pop(0)` - that shifts every element, so it is $O(n)$ per dequeue and $O(n^2)$ overall.

Three correct options:

- **Doubly linked list** - $O(1)$ at both ends, but a pointer per node and poor
- **Circular buffer** - a fixed array with `head` and `tail` indices that wrap via
- **Deque from the standard library** - `collections.deque`, `ArrayDeque`,

The classic puzzle - implement a queue with two stacks - is worth knowing: push onto `in`; to pop, if `out` is empty pour all of `in` into it (reversing order) then pop from `out`. Each element moves at most twice, so it is **$O(1)$ amortised**.

[Medium] Q. What is a monotonic stack and what does it solve?

Answer. A stack kept in sorted order - you pop elements that break the order before pushing. It answers "for each element, what is the next element greater (or smaller) than it?" in **$O(n)$** instead of $O(n^2)$.

```
def next_greater(nums):
    result = [-1] * len(nums)
    stack = []  # holds indices, values decreasing
    for i, n in enumerate(nums):
        while stack and nums[stack[-1]] < n:
            result[stack.pop()] = n  # n is the answer for that index
        stack.append(i)
    return result
# [2, 1, 3] -> [3, 3, -1]
```

Each index is pushed once and popped once, so despite the inner `while` the total work is $O(n)$.

The tell: "next greater / previous smaller element", "largest rectangle in a histogram", "daily temperatures", "trapping rain water". If a problem asks you to relate each element to the nearest element satisfying a comparison, try a monotonic stack.

[Easy] Q. What is a heap, and what is it good at?

Answer. A complete binary tree satisfying the heap property: in a min-heap, every parent is $\leq$ its children (so the minimum sits at the root).

It is stored in a **flat array**, not with pointers - for index i , children are at $2i+1$ and $2i+2$, parent at $(i-1)//2$. That is why heaps are fast: no allocation, excellent cache locality.

Operation	Cost
Peek min/max	$O(1)$
Insert	$O(\log n)$ - sift up
Extract min/max	$O(\log n)$ - sift down
Build from n items	$O(n)$, not $O(n \log n)$
Search for arbitrary value	$O(n)$ - no ordering between siblings

Two points interviewers probe:

- **Building a heap is $O(n)$.** Sifting down from the middle backwards costs less
- **A heap is only partially ordered.** Finding an arbitrary element is $O(n)$, and

[Medium] Q. How do you find the k largest elements in a stream of n items?

Answer. A min-heap of size k .

```
import heapq

def k_largest(stream, k):
    heap = []  # min-heap of the best k so far
    for item in stream:
        if len(heap) < k:
            heapq.heappush(heap, item)
        elif item > heap[0]:  # bigger than the smallest kept
            heapq.heapreplace(heap, item)
    return sorted(heap, reverse=True)
```

$O(n \log k)$ time, $O(k)$ space.

The counter-intuitive part, and the point of the question: to track the k largest you use a **min-heap**. The root is the weakest of your current champions, so it is both the cheapest to check against and the right one to evict.

Why not sort? Sorting is $O(n \log n)$ time and **$O(n)$ space** - impossible if the stream does not fit in memory. When k is small and n is huge, $O(n \log k)$ with $O(k)$ space is the meaningful win.

[Medium] Q. What is a priority queue and how does it differ from a heap?

Answer. A **priority queue** is an abstract data type: "give me the highest-priority item next". A **heap** is the usual concrete implementation.

The distinction matters because other implementations exist with different trade-offs - a sorted array gives $O(1)$ extraction but $O(n)$ insertion; a Fibonacci heap gives $O(1)$ amortised insert and decrease-key, which is what makes the textbook Dijkstra bound $O(E + V \log V)$.

In interviews, "priority queue" almost always means "binary heap". It is the engine inside Dijkstra's algorithm, A*, Huffman coding, and event-driven simulation.

[Medium] Q. How do you check for balanced brackets?

Answer. The canonical stack problem. Push opening brackets; on a closing bracket, the top of the stack must be its partner.

```
PAIRS = {"(": ")", "[": "]", "{": "}"

def is_balanced(s):
    stack = []
    for ch in s:
        if ch in "([{":
            stack.append(ch)
        elif ch in PAIRS:
            if not stack or stack.pop() != PAIRS[ch]:
                return False          # wrong partner, or nothing open
    return not stack                  # anything left open is unbalanced
```

$O(n)$ time, $O(n)$ space. The two edge cases that separate a correct answer from a nearly-correct one: a closing bracket when the stack is **empty**, and leftover openers at the **end**. Candidates routinely forget the second and return `True` for `"((("`.

Trees and Graphs

The highest-yield topic after hash tables. Most "hard" interview problems are a graph problem wearing a costume.

[Easy] Q. What is a binary search tree, and what is its complexity?

Answer. A binary tree where, for every node, all keys in the left subtree are smaller and all keys in the right subtree are larger. That invariant makes search a series of "left or right?" decisions.

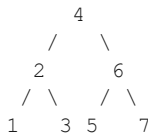
Search, insert and delete are **$O(h)$** where h is the height - **not** $O(\log n)$. The two coincide only when the tree is balanced:

- Balanced: $h \sim \log n \rightarrow O(\log n)$.
- Degenerate (insert already-sorted data): $h = n \rightarrow O(n)$, a linked list with

That degeneration is the entire reason self-balancing trees exist. Saying " $O(h)$, which is $O(\log n)$ only if balanced" is a much stronger answer than " $O(\log n)$ ".

[Easy] Q. What are the tree traversals and when do you use each?

Answer. Three depth-first orders, defined by when you visit the node relative to its children, plus one breadth-first order.

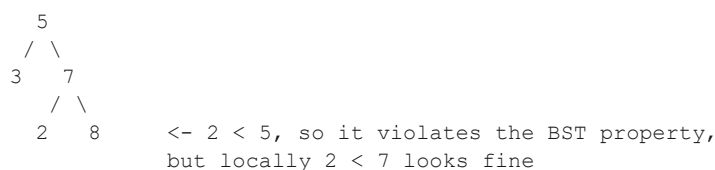


Traversal	Order	Result	Use
In-order	left, node , right	1 2 3 4 5 6 7	Sorted output from a BST
Pre-order	node , left, right	4 2 1 3 6 5 7	Copying/serialising a tree
Post-order	left, right, node	1 3 2 5 7 6 4	Deleting; computing from children up
Level-order (BFS)	by depth	4 2 6 1 3 5 7	Shortest path; level-by-level work

The fact worth memorising: **in-order traversal of a BST yields sorted order**. "Validate a BST" and "find the k th smallest" are both just that observation.

[Medium] Q. How do you validate a binary search tree?

Answer. The trap: checking only `left < node < right` locally is **wrong**. It passes trees that are not BSTs:



Every node must fall inside a range inherited from its ancestors:

```
def is_bst(node, low=float("-inf"), high=float("inf")):
```

```

if node is None:
    return True
if not (low < node.value < high):
    return False
return (is_bst(node.left, low, node.value) and
        is_bst(node.right, node.value, high))

```

Equivalently: do an in-order traversal and check the output is strictly increasing. Both are $O(n)$ time, $O(h)$ space.

[Medium] Q. Which balanced trees should I know, and what balances them?

Answer. You need the idea, not the rotation code.

- **AVL** - strictly balanced; heights of sibling subtrees differ by at most 1.
- **Red-black** - loosely balanced via colour rules; the longest path is at most
- **B-tree / B+ tree** - high branching factor, so the tree is very shallow. Used

All give $O(\log n)$ worst case. The distinction is the constant factor and the read/write balance.

[Easy] Q. BFS or DFS - how do you choose?

Answer.

	BFS	DFS
Structure	Queue	Stack (or recursion)
Visits	Nearest first	Deepest first
Finds shortest path	Yes , in an unweighted graph	No
Space	$O(\text{width})$ - can be huge	$O(\text{depth})$
Natural for	Levels, shortest hops, "minimum steps"	Cycles, topological sort, connectivity, backtracking

Both are $O(V + E)$ time.

The decisive rule: if the question says shortest, fewest, or minimum number of steps in an unweighted graph, use **BFS**. BFS reaches every node by the fewest edges, so the first time it arrives is the shortest way. DFS gives no such guarantee - it may find a long path first.

If edges are weighted, neither works: use **Dijkstra** (non-negative weights) or **Bellman-Ford** (negative weights allowed).

[Medium] Q. How do you represent a graph, and which representation should you pick?

Answer. Two options, chosen by density.

Adjacency list - map each vertex to its neighbours.

```
graph = {"a": ["b", "c"], "b": ["d"], "c": [], "d": []}
```

Space $O(V + E)$. Iterating a vertex's neighbours is $O(\text{degree})$. **The default.**

Adjacency matrix - a $V \times V$ grid where $m[i][j]$ marks an edge. Space $O(V^2)$ regardless of edge count. Edge lookup is $O(1)$; listing neighbours is $O(V)$.

Pick the matrix when the graph is **dense** (E approaches V^2), when you need constant-time "is there an edge?", or for matrix algorithms like Floyd-Warshall. Pick the list otherwise - real graphs are almost always sparse, and $O(V^2)$ space for a million vertices is a terabyte.

[Medium] Q. Why must graph traversal track visited nodes when tree traversal need not?

Answer. Because a tree is by definition acyclic and each node has exactly one parent - you cannot arrive anywhere twice. A general graph can have cycles and multiple paths to the same node, so without a `visited` set traversal loops forever or does exponential redundant work.

```
def dfs(graph, node, visited=None):
    if visited is None:
        visited = set()
    if node in visited:
        return
    visited.add(node)
    for neighbour in graph[node]:
        dfs(graph, neighbour, visited)
```

Mark a node **when you enqueue/visit it**, not when you dequeue it - otherwise the same node can be queued many times before it is ever processed.

[Hard] Q. What is a topological sort and when is it impossible?

Answer. A linear ordering of a directed graph's vertices such that every edge points forward - every task appears after everything it depends on. Build systems, task schedulers and course prerequisites all need it.

It is impossible exactly when the graph contains a cycle, since a cycle means a set of tasks that each depend on the other. So a topological sort doubles as a cycle detector, which is often the real question.

Kahn's algorithm, $O(V + E)$:

- 1 Compute every vertex's in-degree.
- 2 Enqueue all vertices with in-degree 0.
- 3 Pop one, append it to the output, and decrement its neighbours' in-degrees;
- 4 If the output holds fewer than V vertices, **there is a cycle**.

A recursive example over a hierarchy is in `examples/sql/recursive-ctes/org_chart_tree_traversal.sql`.

[Medium] Q. What is a trie and why use one over a hash table?

Answer. A tree keyed by character, where a word's letters spell out the path from the root. Shared prefixes share nodes.

Lookup is **$O(k)$** for a key of length k - independent of how many words are stored, and with no hashing and no collisions.

The reason to choose it over a hash map is **prefix queries**. "All words starting with `pre`" is one walk to the `pre` node followed by a subtree traversal. A hash map cannot do this at all without scanning every key, because hashing deliberately destroys the relationship between similar keys.

That makes tries the structure behind autocomplete, spell-checkers and IP routing tables. The cost is memory: a node per character per distinct prefix.

Tries & Prefix Data Structures

A trie (prefix tree) stores strings so that shared prefixes are stored once. It is the go-to structure when the question mentions "prefix", "autocomplete", or "dictionary word" - and it is the backbone of command completion, spell-checking, and routing tables.

[Easy] Q. What is a trie, and what does it buy you over a hash set of words?

Answer. A trie is a tree where each edge is labelled with a character and a path from the root spells a string. What it buys you is **prefix awareness**: `search`, `insert`, and `starts_with` all cost $O(L)$ in the string length L (not the number of words), and you can enumerate every word with a given prefix by walking the subtree - something a hash set cannot do at all.

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word: str) -> None:
        node = self.root
        for ch in word:
            node = node.children.setdefault(ch, TrieNode())
        node.is_end = True

    def _walk(self, s: str):
        node = self.root
        for ch in s:
            if ch not in node.children:
                return None
            node = node.children[ch]
        return node

    def search(self, word: str) -> bool:
        node = self._walk(word)
        return node is not None and node.is_end

    def starts_with(self, prefix: str) -> bool:
        return self._walk(prefix) is not None

t = Trie()
for w in ["cat", "car", "card", "care"]:
    t.insert(w)
assert t.search("cat") and t.search("card") and not t.search("ca")
assert t.starts_with("car") and not t.starts_with("dog")
```

The key detail: `search("ca")` is `False` even though `starts_with("ca")` is `True`, because `ca` is a prefix but was never itself inserted - the `is_end` flag distinguishes "a word ends here" from "a path passes through here".

[Medium] Q. How do you list all words that share a prefix, and what's the cost?

Answer. Walk to the node for the prefix ($O(L)$), then a DFS of that subtree collecting every `is_end` node. The cost is $O(L + K)$ where K is the total number of characters in all matching words - you pay for the output, which is unavoidable.

```
def words_with_prefix(self, prefix: str) -> list[str]:
    node = self._walk(prefix)
    if node is None:
```

```

        return []
    out = []
    def dfs(n, acc):
        if n.is_end:
            out.append(prefix + acc)
        for ch in sorted(n.children):
            dfs(n.children[ch], acc + ch)
    dfs(node, "")
    return out

t.words_with_prefix("ca")    # ['car', 'card', 'care', 'cat']
t.words_with_prefix("do")    # []

```

This is exactly the autocomplete step. Note the `sorted(...)` makes the output deterministic (alphabetical); drop it if you only care about existence and want to stop early.

[Medium] Q. Trie vs sorted array + binary search vs hash set - when is each right?

Answer.

Need	Best fit	Why
"Is this exact word present?"	Hash set	O(L) expected, tiny constant
"Any word with this prefix?"	Trie	Subtree walk; the other two can't
"Nearest word / suggest close match"	Trie	DFS with a small edit budget
Memory is tight, no prefix queries	Sorted array	No per-node object overhead

A hash set wins on plain membership and uses far less memory. A trie's cost is one node object per distinct prefix, so a dictionary of 100k long words can be memory-heavy - which is why production systems often use a **radix trie** (compress unary chains into a single labelled edge) or a sorted array with binary search when they don't actually need prefix enumeration.

[Hard] Q. How would you implement "longest word in the dict that is a

prefix of `s`", and what's the complexity?

Answer. Insert every dictionary word into a trie, then walk `s` character by character, tracking the deepest `is_end` node you pass. When the walk falls off a branch, stop - you've found the longest prefix that exists. Time is $O(L + \text{total dict characters})$ to build plus $O(L)$ to query; space $O(\text{total dict characters})$.

```

def longest_prefix(trie: "Trie", s: str) -> str:
    node = trie.root
    best = ""
    for i, ch in enumerate(s):
        if ch not in node.children:
            break
        node = node.children[ch]
        if node.is_end:
            best = s[: i + 1]
    return best

t = Trie()
for w in ["we", "word", "flower", "flowering"]:
    t.insert(w)
assert longest_prefix(t, "flowerpowers") == "flower"
assert longest_prefix(t, "dog") == ""

```

The trap is returning the last matched character instead of the last position where a **complete word** ended - "flowe" walks fine but isn't a word, so `is_end` is what makes the answer "flower", not "flowe".

[Easy] Q. Why do we mark a terminal node instead of, say, storing the word?

Answer. Storing the full word at each terminal node would duplicate every prefix's characters and blow the space from $O(\text{total characters})$ to $O(\text{total characters} \times \text{average length})$. A single boolean `is_end` marks the end of a word while the shared path still holds each character exactly once. If you do need the actual string (e.g. to return it), reconstruct it from the path during the walk - that's what `prefix + acc` does in the DFS above - rather than storing copies.

Algorithms

Complexity Analysis

If you get one subject right, make it this one. Every other answer you give will be followed by "and what's the complexity?"

[Easy] Q. What does Big-O actually describe?

Answer. An upper bound on how the cost of an algorithm grows as the input grows, ignoring constant factors and lower-order terms.

It deliberately throws away detail. $O(n)$ says "doubling the input roughly doubles the work". It does not say the algorithm is fast - an $O(n)$ algorithm with a huge constant can lose to an $O(n \log n)$ one at every realistic size.

The three you will be asked to distinguish:

- O - upper bound ("no worse than").
- Ω - lower bound ("no better than").
- Θ - both, a tight bound.

In practice interviewers say "Big-O" but mean Θ . Answer with the tight bound and you will never be wrong.

[Easy] Q. Order these from fastest-growing to slowest: $O(n \log n)$, $O(1)$, $O(2^n)$, $O(n^2)$, $O(\log n)$, $O(n)$

Answer. From slowest-growing (best) to fastest-growing (worst):

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$$

A rough feel for what each survives, at roughly a second of compute:

Complexity	Largest practical n
$O(\log n)$	effectively unbounded
$O(n)$	~100,000,000
$O(n \log n)$	~5,000,000
$O(n^2)$	~10,000
$O(2^n)$	~25
$O(n!)$	~11

This table is the real reason constraints appear in problem statements. If the input is 10^5 , an $O(n^2)$ solution will not pass and you should stop and look for an $O(n \log n)$ one.

[Medium] Q. What is amortised complexity, and why is appending to a dynamic array $O(1)$?

Answer. Amortised complexity is the average cost per operation across a worst-case sequence of operations - not the average over random inputs.

A dynamic array (Python `list`, Java `ArrayList`, Go `slice`, C++ `vector`) keeps spare capacity. Appending is normally a write to a free slot: $O(1)$. When capacity runs out it allocates a bigger block - usually double - and copies everything, which is $O(n)$.

The doubling is what makes it work. To reach n elements the copies cost $1 + 2 + 4 + \dots + n$, which sums to less than $2n$. Spread across n appends that is $O(1)$ each.

The follow-up: what if it grew by a constant 10 instead of doubling? Then reaching n costs $10 + 20 + 30 + \dots$, an arithmetic series that sums to $O(n^2)$, so each append is $O(n)$ amortised. Geometric growth is the whole trick.

[Medium] Q. What is the difference between amortised, average-case and worst-case?

Answer. They answer different questions:

- **Worst case** - the most expensive single input. Hash table lookup: $O(n)$,
- **Average case** - expected cost over a distribution of inputs. Hash table
- **Amortised** - guaranteed average across a sequence, with no probability

The distinction that catches people out: amortised is a guarantee, average is a statistical expectation. A quicksort's $O(n \log n)$ is average-case - an adversary can still force $O(n^2)$. A dynamic array's $O(1)$ append is amortised - no adversary can break it.

[Easy] Q. How do you analyse nested loops?

Answer. Multiply the iteration counts, but read the bounds carefully.

```
for i in range(n):           # n
    for j in range(n):       # n
        work()               # O(1)
# O(n^2)
```

The one that trips people up:

```
for i in range(n):
    for j in range(i, n):    # shrinking inner loop
        work()
```

That is $n + (n-1) + \dots + 1 = n(n+1)/2$, which is still $O(n^2)$. Halving the work does not change the growth class.

And the one that looks quadratic but is not:

```
for i in range(n):
    j = 1
    while j < n:
        j *= 2                # log n iterations, not n
```

That is $O(n \log n)$.

[Medium] Q. What is the complexity of this, and why?

```
def f(s):
    result = ""
    for ch in s:
        result += ch
    return result
```

Answer. $O(n^2)$ in languages with immutable strings (Python, Java, C#, JavaScript).

`result += ch` cannot extend the existing string - strings are immutable, so it allocates a new string and copies every character across. Doing that n times copies $1 + 2 + \dots + n$ characters, which is $O(n^2)$.

The fix is to collect the pieces and join once, which is $O(n)$:

```
return "".join(s)           # Python
# StringBuilder in Java/C#, array.push + join in JavaScript
```

This is a genuinely common interview trap, because the $O(n^2)$ version looks like a simple linear loop.

[Medium] Q. How do you reason about the complexity of a recursive function?

Answer. Count the nodes in the recursion tree, then multiply by the work done per node.

Fibonacci, naive:

```
def fib(n):
    if n <= 1:
        return n
    return fib(n - 1) + fib(n - 2)
```

Each call spawns two, to a depth of n , so roughly 2^n nodes with $O(1)$ work each: **$O(2^n)$ time, $O(n)$ space** (the deepest path of the call stack).

Merge sort: the tree has $\log n$ levels, and every level touches all n elements during the merges: **$O(n \log n)$** .

Binary search: one child per call, $\log n$ deep, $O(1)$ per node: **$O(\log n)$** .

For divide-and-conquer, the Master Theorem formalises this, but the recursion tree gets you there and is easier to explain out loud.

[Medium] Q. What is space complexity, and does the call stack count?

Answer. Space complexity is the extra memory an algorithm needs beyond its input - and yes, the call stack counts.

A recursive function that is $O(n)$ deep uses $O(n)$ space even if it allocates nothing, which is exactly why deep recursion causes stack overflow. Converting to iteration with an explicit stack does not reduce the space, but it does move it from the (small, fixed) stack to the (large) heap.

Common values worth memorising:

Algorithm	Time	Space
Merge sort	$O(n \log n)$	$O(n)$
Quicksort	$O(n \log n)$ avg	$O(\log n)$ stack
Heap sort	$O(n \log n)$	$O(1)$
BFS / DFS on a graph	$O(V + E)$	$O(V)$
Binary search, iterative	$O(\log n)$	$O(1)$

Heap sort being the only $O(1)$ -space $O(n \log n)$ comparison sort is a favourite follow-up.

Dynamic Programming

The topic candidates fear most. It is far more learnable than its reputation - almost every DP problem is the same four steps applied to a different state.

[Easy] Q. What is dynamic programming?

Answer. Solving a problem by breaking it into overlapping subproblems and storing each subproblem's answer so it is computed only once.

Two properties must hold, and naming both is the mark of a real answer:

- 1 **Optimal substructure** - the optimal solution is built from optimal
- 2 **Overlapping subproblems** - the same subproblem recurs many times.

If subproblems don't overlap, you have divide and conquer (merge sort), not DP. If there is no optimal substructure, DP does not apply at all.

[Medium] Q. Memoisation vs tabulation?

Answer. Two ways to implement the same recurrence.

Memoisation (top-down) - ordinary recursion plus a cache.

```
from functools import cache

@cache
def fib(n):
    return n if n <= 1 else fib(n - 1) + fib(n - 2)
```

Tabulation (bottom-up) - fill a table from the base cases up.

```
def fib(n):
    if n <= 1:
        return n
    prev, curr = 0, 1
    for _ in range(n - 1):
        prev, curr = curr, prev + curr
    return curr
```

	Memoisation	Tabulation
Written as	Recursion + cache	Loops
Computes	Only reachable states	Every state
Space	Table + O(depth) call stack	Table, often reducible
Risk	Stack overflow when deep	None
Easier to	Write from the recurrence	Optimise the space of

Write the memoised version first - it follows directly from the recurrence and is much harder to get wrong. Convert to tabulation if you need the space optimisation or are worried about stack depth.

[Medium] Q. How do you recognise a DP problem?

Answer. Signals, in rough order of reliability:

- It asks for a **count** of ways, or a **minimum/maximum**, or whether something
- The brute force is **exponential** and recomputes the same states.
- It involves a **subsequence** (not subarray), or choices of take/skip.
- There are small numeric constraints that look like array dimensions ($n \leq 1000$)

Counter-signal: if a locally optimal choice is provably globally optimal, it is a **greedy** problem, which is simpler. Interval scheduling by earliest finish time is greedy; the knapsack is not.

[Hard] Q. Walk through solving a DP problem from scratch.

Answer. Four steps. Do them in this order, out loud.

Problem: climbing stairs, taking 1 or 2 steps at a time - how many distinct ways to reach step n ?

1. Define the state. $\text{ways}(i)$ = the number of distinct ways to reach step i . Getting this sentence precise is 80% of the work.

2. Find the recurrence. You arrive at step i either from $i-1$ (a 1-step) or from $i-2$ (a 2-step), and those routes are disjoint:

```
ways(i) = ways(i - 1) + ways(i - 2)
```

3. Base cases. $\text{ways}(0) = 1$ (one way to stand still), $\text{ways}(1) = 1$.

4. Order of evaluation. i depends on smaller i , so iterate upward.

```
def climb(n):
    prev, curr = 1, 1                # ways(0), ways(1)
    for _ in range(n - 1):
        prev, curr = curr, prev + curr
    return curr
```

$O(n)$ time, $O(1)$ space - because each state needs only the previous two, the table collapses to two variables. That collapse is the most common DP optimisation and interviewers look for it.

[Hard] Q. Explain the 0/1 knapsack.

Answer. Given items with weights and values and a capacity, maximise value without exceeding capacity. Each item is taken at most once - the "0/1".

State: $\text{best}[i][c]$ = the best value using the first i items with capacity c .

Recurrence - for each item, take it or skip it:

```
best[i][c] = max(
    best[i-1][c],                # skip
    best[i-1][c - weight[i]] + value[i]  # take, if it fits
)

def knapsack(weights, values, capacity):
    best = [0] * (capacity + 1)
    for w, v in zip(weights, values):
        for c in range(capacity, w - 1, -1):  # BACKWARDS
            best[c] = max(best[c], best[c - w] + v)
    return best[capacity]
```

$O(n \times \text{capacity})$ time, $O(\text{capacity})$ space.

The backwards inner loop is the subtle part. Iterating forwards would let the same item be used twice, because `best[c - w]` would already have been updated in this round. Forwards gives you the **unbounded** knapsack (unlimited copies); backwards gives 0/1. Being able to explain that one line is a strong signal.

Is it polynomial? No - this is pseudo-polynomial. `capacity` is a numeric value, not an input size, so the table is exponential in the number of bits used to write the capacity. 0/1 knapsack is NP-hard.

[Medium] Q. What are the standard DP patterns worth recognising?

Answer. Most problems are a variation of one of these.

Pattern	State	Examples
Linear	<code>dp[i]</code> from <code>dp[i-1]</code> , <code>dp[i-2]</code>	Climbing stairs, house robber, max subarray
Two sequences	<code>dp[i][j]</code> over both	Edit distance, longest common subsequence
Knapsack	<code>dp[item][capacity]</code>	Subset sum, coin change, partition
Interval	<code>dp[left][right]</code>	Matrix chain, burst balloons, palindromes
Grid	<code>dp[row][col]</code>	Unique paths, minimum path sum
State machine	<code>dp[i][state]</code>	Stock trading with cooldown or fees

Naming the pattern early ("this is a two-sequence DP, so I want a 2-D table indexed by positions in each string") makes the rest follow mechanically.

[Medium] Q. What is the difference between DP and greedy?

Answer. Greedy commits to the locally best choice and never reconsiders. DP explores all choices and keeps the best.

Greedy is faster (usually $O(n \log n)$ vs $O(n^2)$ or worse) but is only correct when the problem has the **greedy-choice property** - that a locally optimal choice is part of some globally optimal solution.

The classic demonstration is coin change. Greedy (take the largest coin that fits) works for `[1, 5, 10, 25]`: for 30 it gives `25 + 5`, which is optimal. It fails for `[1, 3, 4]`: for 6 it gives `4 + 1 + 1 = three` coins, but `3 + 3 = two` is optimal. Same algorithm, different coin set, wrong answer.

So: if you propose greedy, you owe a justification or a proof sketch. If you cannot give one, use DP. Never assert greedy works because it passes your two hand-picked examples.

Number Theory & Math

The math questions that actually come up: GCD, modular arithmetic, primality, and a handful of bit tricks. None of them require advanced theory - they require knowing which tool fits which shape of problem.

[Easy] Q. How does the Euclidean algorithm compute a GCD, and why is it fast?

Answer. $\text{gcd}(a, b) == \text{gcd}(b, a \% b)$, repeated until the remainder is 0. Each step shrinks the pair, and the remainder at least halves every two steps, so it runs in $O(\log(\min(a, b)))$ - which is why it is the workhorse behind fraction reduction, CRT, and RSA.

```
def gcd(a: int, b: int) -> int:
    while b:
        a, b = b, a % b
    return a

assert gcd(1071, 462) == 21
assert gcd(0, 5) == 5          # gcd(0, x) == x by convention
assert gcd(270, 192) == 6
```

The extended version also returns x, y with $a*x + b*y == \text{gcd}(a, b)$ - that identity is what makes modular inverses possible at all.

[Medium] Q. When do you need modular arithmetic, and how do you divide "mod p"?

Answer. Modulo keeps numbers bounded - hashing, cyclic buffers, "is it a power of two" checks, and big-prime arithmetic in cryptography. The trap is division: $a / b \bmod p$ is NOT $(a \bmod p) / (b \bmod p)$. You multiply by the **modular inverse** of b instead, which exists only when $\text{gcd}(b, p) == 1$ (always true when p is prime and b is not a multiple of it).

```
# "1000 * 7^(mod -1) mod 13" - i.e. 1000/7 mod 13, done by multiplying
# 1000 by the modular inverse of 7.
p = 13
assert pow(7, -1, p) == 2          # 7 * 2 = 14 = 1 (mod 13)
answer = (1000 % p) * pow(7, -1, p) % p
assert answer == 11                # 1000 * 2 = 2000 = 11 (mod 13)
```

When p is prime you can also get the inverse from Fermat's little theorem: $b^{(p-2)} \bmod p$. Python's `pow(base, exp, mod)` does fast modular exponentiation, so this is cheap.

[Medium] Q. Trial division vs a sieve - how do you test or generate primes?

Answer. To test ONE number, trial-divide up to its square root (skip evens after 2): $O(\sqrt{n})$. To generate ALL primes up to n , use the Sieve of Eratosthenes: mark multiples of each prime starting at its square, because smaller multiples were already crossed off. The sieve is roughly $O(n \log \log n)$ and beats testing each number individually by a wide margin once n is large.

```
def is_prime(n: int) -> bool:
    if n < 2:
        return False
    if n < 4:
        return True
    if n % 2 == 0:
        return False
    f = 3
    while f * f <= n:          # only need to test up to sqrt(n)
        if n % f == 0:
            return False
        f += 2
```

```

        f += 2
    return True

assert is_prime(2) and is_prime(3) and is_prime(7)
assert not is_prime(1) and not is_prime(0) and not is_prime(2**20)

def sieve(n: int) -> list[int]:
    prime = bytearray([1]) * (n + 1)
    prime[0:2] = b"\x00\x00"
    for i in range(2, int(n ** 0.5) + 1):
        if prime[i]:
            prime[i * i :: i] = bytearray(len(prime[i * i :: i]))
    return [i for i in range(n + 1) if prime[i]]

assert sieve(30) == [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

```

Starting each marking pass at $i*i$ (not $2*i$) is the detail interviewers probe for: everything below $i*i$ that is a multiple of i already has a smaller prime factor and was crossed off earlier.

[Medium] Q. What are the bit tricks you should be able to write without looking up?

Answer. With n a non-negative int: $n \& 1$ tests parity; $n \& (n-1)$ clears the lowest set bit (and is 0 exactly when n is a power of two); $n | (n-1)$ rounds up to the next all-ones mask; $n \wedge (n-1)$ isolates the run of low bits. For the count of set bits, either loop-and-clear or `bin(n).count("1")`.

```

assert bin(12).count("1") == 2
assert (16 & 15) == 0 and (12 & 11) == 8    # 10000->0, 1100->1000
assert (5 & (5 - 1)) == 4                  # clear lowest set bit: 101 -> 100
assert (7 | (7 - 1)) == 7 and (6 | 5) == 7  # round up to next all-ones
assert ((n := 8) & (n - 1)) == 0           # power of two -> 0

```

The power-of-two test generalises to "is this a valid bitmask / block size?" checks all over systems code. The XOR identity $a \wedge a == 0$ and $a \wedge 0 == a$ is also how you find the single unpaired element in one pass.

[Hard] Q. You need a/b as a fraction reduced to lowest terms, and a and b

are up to 10^{18} . What do you do, and what are the overflow traps?

Answer. Compute $g = \text{gcd}(a, b)$ and return $(a//g, b//g)$. The Euclidean algorithm is safe at that size in Python (arbitrary precision), but in fixed-width languages the trap is not the GCD - it is anywhere you **multiply** before dividing. If a problem needs $a * b / c$ and you do the multiplication first, you can overflow even when the final quotient fits. Reduce by gcd factors before multiplying, or use a bignum / `__int128` / `Math.multiplyHigh` equivalent.

```

from math import gcd

def reduce(a: int, b: int) -> tuple[int, int]:
    g = gcd(a, b)
    return a // g, b // g

assert reduce(462, 1071) == (22, 51)    # gcd is 21
assert reduce(20, 50) == (2, 5)
assert reduce(0, 7) == (0, 1)
assert reduce(10**18, 10**18 - 10**9) == (10**9, 10**9 - 1)

```

The follow-up they will ask: the same for adding fractions $a/b + c/d = (a*d + c*b) / (b*d)$ - the numerator and denominator can be quadratically larger than the inputs, so cross-reduce with $\text{gcd}(a, d)$ and $\text{gcd}(c, b)$ before the big multiply.

[Easy] Q. When is a number a power of two, and how do you find that power?

Answer. A positive integer is a power of two exactly when it has a single set bit: $n > 0$ and $(n \& (n - 1)) == 0$. To find the exponent use the bit length minus one ($n.bit_length() - 1$), which is $O(1)$ in Python.

```
def is_power_of_two(n: int) -> bool:
    return n > 0 and (n & (n - 1)) == 0

assert is_power_of_two(1) and is_power_of_two(1024)
assert not is_power_of_two(0) and not is_power_of_two(12)

assert (64).bit_length() - 1 == 6      # 64 == 2**6
```

Why it works: a power of two looks like $1000\dots0$; subtracting 1 gives $0111\dots1$, so the AND is 0. Any number with two or more set bits keeps at least one bit in common with $n-1$. The $n > 0$ guard matters - 0 is $1000 \& 0111$ -free but is not a power of two.

Recursion and Backtracking

Recursion is how you express "solve a smaller version of this". Backtracking is recursion that undoes its choices - the standard tool for permutations, combinations and constraint puzzles.

[Easy] Q. What are the parts of a correct recursive function?

Answer. Three, and missing any one is a bug:

- 1 **A base case** that returns without recursing.
- 2 **A recursive case** that calls itself on a **strictly smaller** input.
- 3 **Progress** - every path must move toward the base case.

```
def factorial(n):  
    if n <= 1:          # base case  
        return 1  
    return n * factorial(n - 1)  # smaller input, guaranteed progress
```

Infinite recursion is almost always a missing base case or a recursive call that does not shrink the input (`factorial(n)` instead of `factorial(n - 1)`).

[Easy] Q. What actually happens in memory when a function recurses?

Answer. Each call pushes a **stack frame** holding that call's parameters, local variables and return address. Frames are popped as calls return.

That is why recursion costs **O(depth) space** even when it allocates nothing, and why exceeding the stack limit raises `StackOverflowError` / `RecursionError`. Python's default limit is 1000 frames - deliberately low, to turn runaway recursion into a clean error instead of a crash.

The practical consequence: recursion depth proportional to n is fine for a balanced tree (depth $\log n$) and dangerous for a linked list or a degenerate tree (depth n).

[Medium] Q. What is tail recursion, and does it help in your language?

Answer. A call is in **tail position** when it is the very last thing the function does - its result is returned directly, with no pending work.

```
def sum_to(n, acc=0):  
    if n == 0:  
        return acc  
    return sum_to(n - 1, acc + n)  # tail call: nothing waits on it
```

A compiler can reuse the current frame instead of pushing a new one, turning the recursion into a loop: $O(1)$ space. Scheme and Lua guarantee this; Scala, Kotlin and most functional languages do it (`@tailrec`); JavaScript engines mostly do not, despite it being in the spec.

Python does not do it and will not - Guido van Rossum has rejected it deliberately, partly because it destroys the stack traces that make debugging possible. So in Python, rewriting for tail position buys you nothing. Convert to an explicit loop instead.

[Medium] Q. When should you convert recursion to iteration?

Answer. When depth can be large, or when the recursion is simple enough that a loop is clearer.

Every recursion can be made iterative with an explicit stack. That does not reduce total space - it moves it from the limited call stack to the heap, which is exactly the point when depth is the problem.

Prefer **recursion** for trees, nested structures and divide-and-conquer, where it mirrors the shape of the data. Prefer **iteration** for linear traversals, and whenever depth is $O(n)$ on untrusted input.

[Medium] Q. What is backtracking?

Answer. Systematic search: make a choice, recurse, then **undo the choice** before trying the next one. It explores a tree of partial solutions and abandons a branch as soon as it cannot lead anywhere ("pruning").

The template is the same every time:

```
def backtrack(state, choices, results):
    if is_complete(state):
        results.append(state.copy())    # copy! state keeps mutating
        return
    for choice in choices:
        if not is_valid(state, choice):
            continue                    # prune
        state.append(choice)            # 1. choose
        backtrack(state, choices, results) # 2. explore
        state.pop()                     # 3. un-choose
    return results
```

Two bugs account for most failures: **forgetting to undo** (state leaks between branches) and **appending** state **instead of** `state.copy()` (every result is the same list object, which ends up empty).

[Medium] Q. Generate all permutations of a list.

Answer.

```
def permutations(nums):
    results, used, current = [], [False] * len(nums), []

    def backtrack():
        if len(current) == len(nums):
            results.append(current.copy())
            return
        for i, n in enumerate(nums):
            if used[i]:
                continue
            used[i] = True
            current.append(n)
            backtrack()
            current.pop()    # undo
            used[i] = False  # undo
        return

    backtrack()
    return results
```

$O(n \times n!)$ time - there are $n!$ permutations and copying each costs $O(n)$ - and **$O(n)$ space** beyond the output.

You cannot beat $n!$ when the output is $n!$ items; that is an output-size bound, not an inefficiency. Say so explicitly, because candidates often apologise for a complexity that is provably optimal.

[Medium] Q. Subsets vs permutations vs combinations - how do the recursions differ?

Answer. All three are the same skeleton with a different loop bound.

	Count	Key difference
Subsets	2^n	Every element is take-or-skip; record at every node
Permutations	$n!$	Order matters; loop over all unused elements
Combinations (choose k)	$C(n,k)$	Order does not matter; loop from <code>start</code> onward

The single line that separates permutations from combinations is whether the recursive call restarts the loop at 0 (permutations) or at `i + 1` (combinations). Passing a `start` index is what stops `[1,2]` and `[2,1]` both appearing.

[Hard] Q. How does pruning change backtracking's performance?

Answer. It does not improve the worst case, but it is the difference between usable and useless in practice.

N-Queens is the standard illustration. Brute force places a queen in every square of every row: $8^8 \sim 16.7$ million placements for an 8x8 board. Checking validity as you place - abandoning a row the instant a queen is attacked - cuts it to roughly 15,000 nodes explored, and only 92 solutions exist.

The principle: **check constraints as early as possible**, not at the leaves. A branch you cut at depth 2 removes everything beneath it. This is also why the order you try choices matters - trying the most constrained option first ("minimum remaining values") prunes hardest and is the core heuristic in Sudoku solvers.

Selection Algorithms

[Medium] Q. How would you find the kth smallest value without fully sorting?

Answer. Use **quickselect** for expected $O(n)$ time and $O(1)$ auxiliary space when mutation is allowed, or keep a size-k heap for $O(n \log k)$ time when input arrives as a stream or must not be rearranged.

Quickselect partitions around a pivot like quicksort, but continues only into the side containing rank k. Balanced partitions cost $n + n/2 + n/4 + \dots = O(n)$. Repeated bad pivots give an $O(n^2)$ worst case, so real implementations randomize the pivot or use a defensive strategy.

The trade-off is the interview-worthy part: sort for many rank queries, use a heap for streaming top-k, and quickselect for a one-off query on a mutable in-memory array.

Sorting and Searching

You will rarely implement a sort in production. You will frequently be asked to compare them, and binary search is one of the most bug-prone things in computing.

[Easy] Q. Compare the sorting algorithms you know.

Answer.

Algorithm	Best	Average	Worst	Space	Stable?
Bubble / insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quicksort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No
Counting / radix	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(n + k)$	Yes

The three sentences worth having ready:

- **Merge sort** guarantees $O(n \log n)$ and is stable, but needs $O(n)$ scratch space.
- **Quicksort** is usually fastest in practice - great locality, in-place - but
- **Heap sort** is the only one that is both $O(n \log n)$ worst case and $O(1)$

[Medium] Q. If quicksort has an $O(n^2)$ worst case, why is it the default almost everywhere?

Answer. Because the worst case is avoidable and the average case is excellent.

$O(n^2)$ happens when the pivot is consistently the smallest or largest element - classically, choosing `arr[0]` on already-sorted input, which is depressingly common in real data. The fixes:

- **Randomised pivot** - an adversary cannot predict it.
- **Median-of-three** - sample first, middle, last.
- **Introsort** - start with quicksort, count recursion depth, and switch to heap

Quicksort also wins on constants: it partitions in place with sequential memory access, so it is very cache-friendly, whereas merge sort writes to a second array.

The real-world answer: most standard libraries no longer use plain quicksort. Python and Java (for objects) use **Timsort**, a stable merge sort that detects already-sorted runs and is $O(n)$ on nearly-sorted data. Java uses dual-pivot quicksort for primitives, where stability is meaningless.

[Medium] Q. What does "stable" mean and when does it matter?

Answer. A stable sort preserves the relative order of elements that compare equal.

It matters whenever you sort **more than once**. To sort employees by department, and by salary within each department, a stable sort lets you do it in two passes:

```
people.sort(key=lambda p: p.salary)      # secondary key first
people.sort(key=lambda p: p.department)  # primary key second
```

The second sort preserves the salary ordering inside each department - only because it is stable. With an unstable sort this silently produces wrong output.

Stability is also why Python's `sorted` and Java's `Collections.sort` guarantee it in their specification: users depend on this pattern.

[Medium] Q. Can you sort faster than $O(n \log n)$?

Answer. Not with **comparisons**. There are $n!$ possible orderings, and each comparison yields one bit, so you need at least $\log_2(n!) \sim n \log n$ comparisons. That is a proven lower bound, not a limitation of current algorithms.

You can beat it by not comparing. **Counting sort** tallies occurrences of each value into buckets: $O(n + k)$ for values in a range of size k . **Radix sort** applies counting sort digit by digit: $O(d \times (n + k))$.

The catch is the constraints. Counting sort on 32-bit integers needs a 4-billion-entry array. These are worth it only when the key range is small and known - sorting ages, exam scores, or fixed-length strings.

If an interviewer asks for better than $O(n \log n)$, they are asking you to notice a constraint on the values.

[Easy] Q. Write binary search. What are the classic bugs?

Answer.

```
def binary_search(arr, target):
    low, high = 0, len(arr) - 1          # inclusive bounds
    while low <= high:                  # <=, not <
        mid = low + (high - low) // 2    # overflow-safe
        if arr[mid] == target:
            return mid
        if arr[mid] < target:
            low = mid + 1                 # must be +1
        else:
            high = mid - 1                # must be -1
    return -1
```

$O(\log n)$ time, $O(1)$ space. Four bugs live in those eight lines:

- 1 `while low < high` instead of `<=` - misses a one-element range, so the
- 2 `(low + high) // 2` - overflows in fixed-width languages (C, Java) when
- 3 `low = mid` instead of `mid + 1` - infinite loop, because the range stops
- 4 **Forgetting the array must be sorted.** Binary search on unsorted data

Test with: empty array, one element, target absent, target at index 0, target at the last index.

[Medium] Q. How do you find the first occurrence of a value in an array with duplicates?

Answer. Do not stop at the first match - record it and keep searching left.

```
def first_occurrence(arr, target):
    low, high = 0, len(arr) - 1
    found = -1
    while low <= high:
        mid = low + (high - low) // 2
        if arr[mid] == target:
            found = mid                    # candidate
            high = mid - 1
```

```

        high = mid - 1                # but look further left
    elif arr[mid] < target:
        low = mid + 1
    else:
        high = mid - 1
    return found

```

Still $O(\log n)$. Mirror it (`low = mid + 1` on a match) for the last occurrence. This "keep searching after you find it" shape is the basis of `bisect_left` / `lower_bound`, and it generalises to the more important question below.

[Hard] Q. What is "binary search on the answer"?

Answer. Binary search does not require an array. It requires a **monotonic predicate**: a boolean function that is false, false, ..., false, true, true, ..., true over the search range. Then you can binary search for the boundary.

This unlocks a whole class of optimisation problems - "minimise the maximum X", "what is the smallest capacity that works?":

```

def min_feasible(low, high, works):
    """Smallest value in [low, high] for which works(value) is True."""
    best = -1
    while low <= high:
        mid = low + (high - low) // 2
        if works(mid):
            best = mid
            high = mid - 1          # try to do better
        else:
            low = mid + 1          # not enough
    return best

```

"Ship packages within D days", "minimum eating speed", "split array to minimise the largest sum" are all this function with a different `works`.

The tell: the problem asks for a minimum or maximum value, the answer lies in a numeric range, and checking "is value V feasible?" is easy even though finding the best V directly is not. Complexity becomes $O(\log(\text{range}) \times \text{cost of works})$.

A worked binary search with insertion points is in

`examples/python/algorithms/binary_search_iterative.py`.

Databases

Covering Indexes

[Medium] Q. What is a covering index, and when can it make a query much faster?

Answer. A covering index contains every column a query needs for filtering, joining, and output, so the database can answer from the index without looking up the base table rows.

For `SELECT created_at FROM orders WHERE customer_id = ?`, an index on `(customer_id, created_at)` may cover the query. It avoids a random table-page lookup for every match and can be a large win for selective, read-heavy paths.

It is not free: extra columns enlarge the index, reduce cache density, and make inserts and updates more expensive. Verify an index-only scan in the query plan, and remember that MVCC visibility rules may still force table access.

Database Normalization

Normalization is about arranging tables so that each **fact** is stored in exactly one place. The payoff is not academic purity - it is the elimination of update, insertion, and deletion **anomalies**, where one logical change has to be written to (or breaks) several rows.

[Easy] Q. What is a functional dependency, and why is it the root of normalization?

Answer. A functional dependency $x \rightarrow y$ means the value of x determines the value of y - two rows with the same x must have the same y . Normalization is the process of splitting tables so that every non-key attribute depends on the **whole key, nothing but the key, and nothing but the key** (the 3NF slogan). Every normal form is a rule about which dependencies are allowed to live in a table.

```
# An unnormalized "enrollment" table where the teacher is repeated per row.
rows = [
    ("S1", "Math", "Alice"),
    ("S1", "Bio", "Bob"),
    ("S2", "Math", "Alice"),    # "Math is taught by Alice" stored TWICE
]
teachers_for_math = [r[2] for r in rows if r[1] == "Math"]
assert teachers_for_math == ["Alice", "Alice"]    # redundant copies
```

The fact "Math is taught by Alice" depends only on `course`, not on the `(student, course)` key - that is the dependency that normalization will eventually force out of the table.

[Medium] Q. Walk through 1NF, 2NF, and 3NF, and name the anomaly each one removes.

Answer.

- **1NF** - atomic values only, no repeating groups / nested arrays.
- **2NF** - 1NF + no partial dependency: every non-key attribute depends
- **3NF** - 2NF + no transitive dependency: non-key attributes depend on

```
import sqlite3

con = sqlite3.connect(":memory:")
cur = con.cursor()
# Unnormalized: (student, course) key, teacher transitively dependent.
cur.execute(
    "CREATE TABLE enrollment "
    "(id INTEGER PRIMARY KEY, student TEXT, course TEXT, teacher TEXT)"
)
cur.executemany(
    "INSERT INTO enrollment (student, course, teacher) VALUES (?, ?, ?)",
    [("S1", "Math", "Alice"), ("S1", "Bio", "Bob"), ("S2", "Math", "Alice")]
)
# The "update anomaly": one logical fact lives in two rows.
cur.execute("SELECT COUNT(*) FROM enrollment WHERE teacher = 'Alice'")
assert cur.fetchone()[0] == 2

# 3NF: split out the transitive dependency course -> teacher.
cur.execute("CREATE TABLE course (course TEXT PRIMARY KEY, teacher TEXT)")
cur.execute("CREATE TABLE takes (student TEXT, course TEXT, PRIMARY KEY (student, course))")
cur.execute("INSERT INTO course VALUES ('Math', 'Alice'), ('Bio', 'Bob')")
cur.execute("INSERT INTO takes VALUES ('S1', 'Math'), ('S1', 'Bio'), ('S2', 'Math')")
cur.execute()
```

```

"SELECT t.student, t.course, c.teacher "
"FROM takes t JOIN course c ON t.course = c.course ORDER BY 1,2"
)
assert cur.fetchall() == [
    ("S1", "Bio", "Bob"), ("S1", "Math", "Alice"), ("S2", "Math", "Alice"),
]

```

Now "Math is taught by Alice" is stored **once** in `course`; updating it touches one row, and enrolling a student never forces you to restate the teacher.

[Medium] Q. What is BCNF, and how is it stronger than 3NF?

Answer. BCNF says: for every non-trivial dependency $X \rightarrow Y$, X must be a **superkey**. 3NF allows a non-key to determine another non-key as long as the dependent side is a key; BCNF forbids it entirely. In practice the two coincide unless you have overlapping candidate keys.

The classic 3NF-but-not-BCNF example: a table `student, course, teacher` with candidate keys `(student, course)` and `(course, teacher)`, where `course → teacher`. Here `teacher` is part of a candidate key, so 3NF is satisfied (the dependent attribute is a key attribute), but `course → teacher` has a non-superkey determinant, so BCNF is violated. Splitting on `course → teacher` (a `course(teacher)` table) restores BCNF.

The interview takeaway: **3NF is the usual stopping point for OLTP schemas** because BCNF decompositions can sometimes lose a dependency and force joins; you reach for BCNF when you specifically want to kill every remaining redundancy.

[Medium] Q. If normalization is so good, why do people denormalize at all?

Answer. Normalization optimises for **write** consistency (no anomalies, minimal storage). Denormalization trades that away to optimise **read** cost - fewer joins, faster reports, simpler queries. It is a deliberate, documented decision, not laziness.

Reason to denormalize	Example
Join-heavy read path is the bottleneck	Store <code>order.total</code> instead of summing <code>line_item</code> each time
Historical / audit value	Keep <code>customer.name_at_purchase</code> frozen at the moment of sale
Redundant fact is cheap and almost never changes	Store <code>country</code> next to a normalised <code>region_id</code>
Analytics / reporting on a copy	A warehouse or materialised view is a denormalised projection

The discipline: identify the specific read you are optimising, store the redundant value, and own the **write path** that keeps it in sync (trigger, application code, or a rebuild job). The moment you denormalise, you re-introduce an update anomaly on purpose - so you must have a mechanism and a test that proves the copy stays consistent.

[Easy] Q. What is a surrogate key vs a natural key, and which do you use?

Answer. A **natural key** is a real-world identifier (email, VIN, passport number); a **surrogate key** is an artificial, meaningless identifier (auto-increment int, UUID) whose only job is to be unique and stable. Surrogates are preferred for primary keys because natural keys change (someone updates their email), are often long (bad for index size and join speed), and can collide or be reused across contexts.

The rule of thumb: use a **surrogate** primary key, keep the natural identifier as a **unique, indexed** column, and foreign keys reference the surrogate. This keeps joins cheap and insulates your schema from the day the

business decides the "real" identifier was a bad choice.

Indexing and Transactions

The two database topics that separate "can write a query" from "can run a system".

[Easy] Q. What is an index and what does it cost?

Answer. A separate data structure - almost always a **B+ tree** - that maps column values to row locations, so the engine can seek instead of scanning every row.

It turns an $O(n)$ table scan into roughly $O(\log n)$.

The costs, which matter as much as the benefit:

- **Writes get slower.** Every `INSERT`, `UPDATE` and `DELETE` must also update
- **Storage.** An index on a wide column can approach the size of the table.
- **They can be ignored.** If a query would match most of the table, the planner

So: index columns you filter, join or sort on frequently. Do not index everything.

[Medium] Q. Why a B+ tree rather than a hash index or a binary tree?

Answer. Because disk and page size dominate.

A B+ tree node holds hundreds of keys and fills one page, so the tree is very shallow - three or four levels indexes millions of rows, meaning three or four page reads. A binary tree with the same data would be ~20 levels deep and each level is a potential random I/O.

B+ trees also keep all data in the **leaves, linked in sorted order**, which makes range scans (`WHERE date BETWEEN ...`, `ORDER BY`) a straight walk along the leaf chain.

A **hash index** gives $O(1)$ equality lookup but supports no range queries, no sorting and no prefix matching. Some engines offer them; the B-tree is the default because it is good at everything rather than excellent at one thing.

[Medium] Q. What is a composite index, and why does column order matter?

Answer. An index on several columns, sorted by the first column, then the second within that, and so on - like a phone book sorted by surname then forename.

This is the **leftmost prefix rule**. With `INDEX (last_name, first_name)`:

Query	Uses the index?
<code>WHERE last_name = 'Khan'</code>	[yes] Yes
<code>WHERE last_name = 'Khan' AND first_name = 'Sara'</code>	[yes] Yes
<code>WHERE first_name = 'Sara'</code>	[no] No

You cannot look someone up by forename in a phone book; same reason. Put the column you always filter on first, and the highest-selectivity column early.

[Medium] Q. Name things that stop an index being used.

Answer. Mostly, wrapping the indexed column in something:

- **A function on the column.** `WHERE YEAR(created_at) = 2026` cannot use an
- **A leading wildcard.** `LIKE '%son'` cannot seek; `LIKE 'John%'` can.
- **Implicit type conversion.** Comparing an indexed `VARCHAR` to a number makes
- **Low selectivity.** An index on a boolean that is 90% true will be skipped.
- **OR across different columns.** Often forces a scan; `UNION` of two indexed

Use `EXPLAIN` (or `EXPLAIN ANALYZE`) to check rather than guessing - look for `Seq Scan / type: ALL` where you expected a seek.

[Easy] Q. What does ACID stand for?

Answer.

- **Atomicity** - all of a transaction's changes apply, or none do.
- **Consistency** - a transaction moves the database from one valid state to
- **Isolation** - concurrent transactions do not see each other's partial work.
- **Durability** - once committed, changes survive a crash (via the write-ahead

Atomicity is the one people can illustrate: a bank transfer debits one account and credits another, and a crash in between must not leave money destroyed.

[Hard] Q. Explain the isolation levels and the anomalies they prevent.

Answer. Stronger isolation removes more anomalies and costs more concurrency.

Level	Dirty read	Non-repeatable read	Phantom read
Read uncommitted	possible	possible	possible
Read committed	prevented	possible	possible
Repeatable read	prevented	prevented	possible*
Serializable	prevented	prevented	prevented

The anomalies:

- **Dirty read** - you read another transaction's uncommitted change, which may
- **Non-repeatable read** - you read the same **row** twice and get different
- **Phantom read** - you run the same **query** twice and get different rows,

* PostgreSQL's repeatable read uses snapshot isolation and does prevent phantoms; the SQL standard does not require this. Defaults differ too - PostgreSQL and Oracle default to read committed, MySQL/InnoDB to repeatable read. Knowing that defaults vary is a better answer than reciting one vendor's.

[Medium] Q. What is a deadlock and how do you handle it?

Answer. Two transactions each hold a lock the other needs, so neither can proceed. Transaction A locks row 1 and wants row 2; B locks row 2 and wants row 1.

Databases **detect** deadlocks (a cycle in the wait-for graph) and resolve them by killing one transaction - the "victim" - which then gets an error and must retry.

Prevention, in order of usefulness:

- 1 **Consistent lock ordering.** If every transaction touches rows in ascending
 - 2 **Keep transactions short.** Never hold a lock across a network call or user
 - 3 **Lower isolation** where the workload permits it.
 - 4 **Retry with backoff.** Deadlocks are normal under load; application code
-

[Medium] Q. Optimistic vs pessimistic locking?

Answer.

Pessimistic - take the lock before reading (`SELECT ... FOR UPDATE`) and hold it until commit. Nobody else can interfere. Correct, but it serialises access and risks deadlocks. Right when contention is high and conflicts are likely.

Optimistic - take no lock. Read a version number with the row, and on write check the version is unchanged:

```
UPDATE accounts SET balance = 500, version = version + 1
WHERE id = 7 AND version = 3;          -- 0 rows updated means someone else won
```

If zero rows are affected, someone else committed first; re-read and retry. Right when conflicts are rare - no locks, better throughput, at the cost of occasional wasted work.

[Medium] Q. Normalisation vs denormalisation?

Answer. Normalisation removes redundancy by splitting data into related tables. Each fact lives in exactly one place, so updates cannot leave contradictory copies. The cost is more joins.

Denormalisation deliberately duplicates data to avoid those joins - faster reads, but every copy must be kept in sync, and that synchronisation is where bugs live.

The practical answer: **normalise by default, denormalise with evidence.** Start at third normal form, measure, and denormalise the specific read path that is demonstrably too slow. Analytics and reporting systems (star schemas) are routinely denormalised because they are read-heavy and loaded in batches.

SQL Query Questions

SQL rounds are mostly the same eight ideas. Know joins, grouping, window functions and how NULL behaves, and you will handle almost anything asked.

[Easy] Q. Explain the join types.

Answer. A join matches rows from two tables on a condition; the type decides what happens to rows that find no match.

Join	Keeps
INNER	Only rows matching in both
LEFT	All left rows; NULLs where the right has no match
RIGHT	All right rows; NULLs where the left has no match
FULL OUTER	All rows from both sides
CROSS	Every combination (Cartesian product)

The **anti-join** is the pattern worth having ready - "rows on the left with no match on the right":

```
SELECT c.*
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id
WHERE o.id IS NULL;      -- the join produced no match
```

Worked examples: [examples/sql/joins/inner_left_right_and_anti_joins.sql](#).

[Medium] Q. What is the difference between WHERE and HAVING?

Answer. WHERE filters rows before grouping. HAVING filters groups after aggregation.

```
SELECT department, COUNT(*) AS headcount
FROM employees
WHERE active = TRUE          -- drop inactive people first
GROUP BY department
HAVING COUNT(*) > 5;        -- then drop small departments
```

You cannot put `COUNT(*) > 5` in WHERE, because at that point the groups do not exist yet. The logical order of evaluation explains every question of this shape:

```
FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY -> LIMIT
```

SELECT running so late is also why you generally cannot reference a column alias in WHERE, but usually can in ORDER BY.

[Medium] Q. What is the trap with NULL?

Answer. NULL means unknown, not empty, and comparisons with unknown are themselves unknown - not false.

- `NULL = NULL` is **NULL**, not true. Use `IS NULL`.
- `WHERE status != 'shipped'` **excludes rows where status is NULL**, which
- Aggregates skip NULLs: `COUNT(col)` ignores them, `COUNT(*)` does not. `AVG`

The worst one, and a genuine favourite in interviews:

```
SELECT * FROM employees
WHERE id NOT IN (SELECT manager_id FROM departments);
```

If any `manager_id` is `NULL`, this returns **zero rows**, always. `x NOT IN (1, NULL)` evaluates to `NOT (x=1 OR x=NULL) = NOT (false OR unknown) = unknown`. Use `NOT EXISTS`, which handles `NULL` correctly. See [examples/sql/subqueries/exists_in_and_correlated.sql](#).

[Medium] Q. Find the second-highest salary.

Answer. Several ways; the interviewer usually wants to see you handle ties and the "no such row" case.

```
-- Ranking approach: DENSE_RANK treats tied salaries as one rank.
SELECT DISTINCT salary
FROM (SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
      FROM employees) ranked
WHERE rnk = 2;

-- Without window functions:
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

The second form returns `NULL` rather than no rows when everyone earns the same - worth saying out loud, because "what if there is no second-highest?" is the follow-up.

[Medium] Q. Explain RANK, DENSE_RANK and ROW_NUMBER.

Answer. All three number rows within a window; they differ on ties.

For salaries 100, 90, 90, 80:

Salary	ROW_NUMBER	RANK	DENSE_RANK
100	1	1	1
90	2	2	2
90	3	2	2
80	4	4	3

- `ROW_NUMBER` - always distinct, ties broken arbitrarily.
- `RANK` - ties share a rank, then it **skips**.
- `DENSE_RANK` - ties share a rank, no gaps.

Use `ROW_NUMBER` for deduplication (keep the first row per group), `DENSE_RANK` for "top 3 salary levels". Worked example: [examples/sql/window-functions/rank_vs_dense_rank_vs_row_number.sql](#).

[Medium] Q. How do you remove duplicate rows, keeping the newest per group?

Answer. Number rows within each group, then keep number 1.

```
WITH ranked AS (
  SELECT id, email, updated_at,
         ROW_NUMBER() OVER (PARTITION BY email ORDER BY updated_at DESC) AS rn
  FROM users
)
DELETE FROM users
WHERE id IN (SELECT id FROM ranked WHERE rn > 1);
```

The CTE is required because a window function cannot appear in `WHERE` - `WHERE` is evaluated before window functions are. Full example: examples/sql/window-functions/deduplicate_keep_newest_per_group.sql.

[Medium] Q. What is a CTE, and when would you use a recursive one?

Answer. A Common Table Expression is a named subquery defined with `WITH`, used to give a step a name and to reference it more than once. It makes long queries readable and is often the only way to use a window function in a filter.

A **recursive** CTE references itself, which is how you traverse hierarchies - org charts, category trees, bill-of-materials, graph reachability:

```
WITH RECURSIVE chain AS (  
    SELECT id, name, manager_id, 1 AS depth  
    FROM employees WHERE manager_id IS NULL      -- anchor  
    UNION ALL  
    SELECT e.id, e.name, e.manager_id, c.depth + 1  
    FROM employees e JOIN chain c ON e.manager_id = c.id  -- recursive step  
)  
SELECT * FROM chain ORDER BY depth;
```

Always confirm the recursion terminates - a cycle in the data loops forever unless you track visited nodes or cap the depth. Worked example: examples/sql/recursive-ctes/org_chart_tree_traversal.sql.

[Medium] Q. What is the difference between `UNION` and `UNION ALL`?

Answer. `UNION` removes duplicate rows; `UNION ALL` keeps everything.

Deduplication requires a sort or hash over the combined result, so `UNION` is materially more expensive. **Use `UNION ALL` unless you actually need distinct rows** - this is one of the most common easy wins in slow queries.

Core Computer Science

Concurrency

Where correct-looking code is wrong 1% of the time, on a machine you cannot reproduce.

[Easy] Q. What is a race condition?

Answer. When the result depends on the unpredictable timing of concurrent operations.

The canonical example is that `count += 1` is not one operation. It is three:

```
LOAD  count -> register
ADD    1
STORE register -> count
```

Two threads can both load 5, both compute 6, and both store 6. Two increments, one net effect - a **lost update**. Worked demonstration: `examples/rust/concurrency/threads_and_channels.rs`.

The general term is a **critical section**: a region that must not be entered by more than one thread at a time.

[Easy] Q. What is a mutex, and how does it differ from a semaphore?

Answer. A **mutex** enforces mutual exclusion: one holder at a time, and ownership matters - the thread that locks it must be the one that unlocks it.

A **semaphore** is a counter permitting **N** concurrent holders. `acquire` decrements (blocking at zero), `release` increments. A semaphore with `N=1` resembles a mutex but has no ownership, so any thread may release it - which makes it usable as a signal between threads, and easier to misuse.

Use a mutex to protect shared state. Use a semaphore to limit a resource pool ("at most 10 concurrent database connections").

[Medium] Q. What is a deadlock in application code, and how do you avoid it?

Answer. Two threads each holding a lock the other needs.

```
# Thread 1          # Thread 2
with lock_a:        with lock_b:
    with lock_b:    with lock_a:    # both block forever
    ...             ...
```

The reliable fix is **consistent lock ordering** - every thread acquires locks in one global order (say, by object id), which makes a cycle impossible. Also: prefer a single coarser lock over two fine ones, use timeouts (`tryLock`) so a deadlock becomes a recoverable error, and never hold a lock across a network call.

[Medium] Q. What does `volatile` / `atomic` mean, and is it enough?

Answer. They solve two different problems, and conflating them is the common error.

Visibility. Without synchronisation, a value written by one thread may sit in a CPU cache or register and never become visible to another. Java's `volatile` forces reads and writes to go to main memory. It does **not** make compound operations safe - `volatile int count; count++` is still a race.

Atomicity. An atomic operation completes indivisibly. `AtomicInteger.incrementAndGet()` performs the whole read-modify-write as one uninterruptible step, usually via a CPU compare-and-swap instruction.

So: `volatile` for a flag one thread writes and others read. **Atomics** for counters. A **mutex** when you must keep several variables consistent with each other - atomics protect one variable each, not an invariant across two.

[Medium] Q. Async/await vs threads - when do you use which?

Answer. They solve different bottlenecks.

Threads give you parallelism across cores. Right for **CPU-bound** work. Costly: ~1 MB of stack each, plus context switches.

Async gives you concurrency on (usually) one thread by suspending a task at its I/O points so another can run. Right for **I/O-bound** work - thousands of in-flight requests, each mostly waiting. Cheap: a suspended task is a small heap object, not a stack.

The rule: **async for waiting, threads for computing**. And the trap that follows from it - a blocking or CPU-heavy call inside an async function stalls the whole event loop and every other task on it. Offload that work to a thread pool.

[Medium] Q. What is the difference between a thread pool and creating threads on demand?

Answer. A pool creates a fixed set of worker threads once and feeds them tasks from a queue.

It fixes two problems with thread-per-task: **creation cost** (spawning a thread is expensive, and doing it per request wastes most of the work) and **unbounded growth** (10,000 simultaneous requests creating 10,000 threads will exhaust memory and thrash the scheduler).

The pool size is the tuning knob: roughly the core count for CPU-bound work (more threads just add switching); much higher for I/O-bound work, since most threads are blocked. A bounded task queue matters too - an unbounded one converts an overload into an out-of-memory error instead of applying backpressure. Worked example: `examples/go/concurrency/worker_pool.go`.

[Hard] Q. What is the producer-consumer problem?

Answer. Producers add items to a shared bounded buffer, consumers remove them. The coordination requirements:

- Producers must **block when the buffer is full** (backpressure).
- Consumers must **block when it is empty**.
- Mutations of the buffer must be mutually exclusive.

Implemented classically with a mutex plus two condition variables, or in modern code with a **blocking queue** (`BlockingQueue`, `queue.Queue`, a Go channel), which packages exactly this.

The point of the question is usually backpressure: if producers never block, a fast producer and a slow consumer grow the buffer until memory runs out. A bounded queue converts that failure into a slowdown, which is almost always what you want.

[Medium] Q. What is thread-safety, and how do you achieve it without locks?

Answer. Code is thread-safe if it behaves correctly when called concurrently, with no external synchronisation required.

Locks are one route. The others are usually better:

- **Immutability.** An object that never changes cannot be raced on. This is the
- **Confinement.** Do not share the state at all - give each thread its own copy
- **Atomics** for single-variable counters and flags.
- **Concurrent collections** - `ConcurrentHashMap` and friends, which lock

Rust enforces this at compile time: its borrow checker rejects a data race rather than letting you find it in production. See `examples/rust/ownership/count_word_frequencies_borrowed.rs`.

[Hard] Q. You use a compare-and-swap (CAS) loop to update a shared pointer

without a lock. A colleague says it's still racy. When would they be right?

Answer. They are right because of the **ABA problem**: CAS only compares the value, not the history. If another thread changes the pointer from A to B and back to A, your CAS sees "still A" and succeeds - even though the object it points at has been replaced (and possibly freed) in between. The value is the same, but the state your update assumed no longer is.

The canonical setting is a **lock-free stack**:

- 1 You read the head, it points to node **A** (A -> B -> C).
- 2 Another thread pops A and B, then a new allocation happens to reuse the
- 3 Your CAS compares "head is still A?" - yes - and links the new node after

The value (the address A) is identical; only the identity of what it points to changed. CAS cannot tell the difference.

The fixes, in order of how often they're the real answer:

- **Version / sequence stamp (the standard fix).** Store the pointer and a
- **Hazard pointers / epoch-based reclamation.** Prevent the real danger
- **Just take a lock.** Be honest about it. ABA is a cost you pay to be

The follow-up this sets up: "lock-free" and "wait-free" are different. A CAS loop can livelock (always losing the race to another thread) even if it never deadlocks - progress guarantees are a spectrum, and ABA is the clearest example of why the memory model, not just the algorithm, is what you actually have to get right.

Security & Cryptography

Interviewers use crypto questions to see if you can separate the three jobs that get lumped together: **confidentiality** (encryption), **integrity and authenticity** (hashes, MACs, signatures), and **availability** (DoS, rate limits). Getting the labels straight is most of the answer.

[Easy] Q. Symmetric vs asymmetric encryption - what's the difference and

why do we need both?

Answer. Symmetric (AES, ChaCha20) uses one shared key for encrypt and decrypt - fast, but you must get the key to the other party safely. **Asymmetric** (RSA, X25519) uses a public/private key pair - anyone can encrypt with the public key, only the holder of the private key can decrypt - which solves key distribution but is far slower.

Real systems use both: TLS does an **asymmetric** (or Diffie-Hellman) key exchange to agree on a fresh **symmetric** session key, then bulk-encrypts the data with the symmetric key. You get the secure key exchange of public keys and the speed of symmetric crypto. The same split underpins signing (sign fast with a hash, then apply the slow private key to the digest).

[Medium] Q. A cryptographic hash is not a MAC. What's the difference, and

why does it matter?

Answer. A plain hash (SHA-256) is a **one-way fingerprint**: it has no secret, so anyone who can see the message can recompute the digest. A **MAC** (HMAC) additionally takes a **shared secret key**, so only someone with the key can produce a valid tag - that's what gives it authenticity.

```
import hashlib, hmac, secrets

# Plain hash: deterministic, and reproducible by ANYONE who has the message.
assert hashlib.sha256(b"password").hexdigest() == \
    hashlib.sha256(b"password").hexdigest()

# HMAC: needs the key to forge a valid tag.
key = secrets.token_bytes(32)
msg = b"transfer 100 to bob"
tag = hmac.new(key, msg, hashlib.sha256).hexdigest()
assert tag == hmac.new(key, msg, hashlib.sha256).hexdigest() # same key, same tag
assert tag != hmac.new(key, b"transfer 200 to bob", hashlib.sha256).hexdigest()
assert hmac.compare_digest(tag, tag) and not hmac.compare_digest(tag, "x")
```

The concrete failure of `hash(secret + msg)` (a "homebrew MAC"): the concatenation is ambiguous, so different `(secret, msg)` pairs collide.

```
# Two different (secret, msg) splits hash to the SAME value:
a = hashlib.sha256(b"sec" + b"retmsg").hexdigest()
b = hashlib.sha256(b"secret" + b"msg").hexdigest()
assert a == b # both are the byte string 'secretmsg' - the split is unknowable
```

HMAC's structure (keyed inner/outer hash) is designed to avoid exactly this. Also note `hmac.compare_digest` - a **timing-safe** comparison, because a naive `==` can leak how many leading bytes of a forger's tag were correct.

[Medium] Q. How should you store user passwords?

Answer. Never a plain hash. Use a **slow, salted, keyed** function: a **per-user random salt** (so identical passwords produce different digests and rainbow tables are useless) plus a deliberately expensive function (bcrypt, scrypt, or Argon2 - or PBKDF2 with a high iteration count) so brute force is expensive.

```
import hashlib, secrets

password = b"hunter2"
salt = secrets.token_bytes(16) # per-user, stored alongside
digest = hashlib.pbkdf2_hmac("sha256", password, salt, 600_000).hex()

# Verification re-derives with the SAME salt and compares in constant time.
import hmac
assert hmac.compare_digest(
    hashlib.pbkdf2_hmac("sha256", password, salt, 600_000).hex(), digest
)
assert not hmac.compare_digest(
    hashlib.pbkdf2_hmac("sha256", b"hunter3", salt, 600_000).hex(), digest
)
# Same password, different salt -> different digest (defeats rainbow tables).
salt2 = secrets.token_bytes(16)
assert hashlib.pbkdf2_hmac("sha256", password, salt2, 600_000).hex() != digest
```

The three properties to name: **salt** (uniqueness), **slowness/work factor** (costs the attacker), and **you cannot reverse it** (so on breach an attacker still has to brute-force each password individually). Store only salt + digest, never the password or a reversible-encrypted form.

[Medium] Q. What happens during a TLS handshake, in one paragraph?

Answer. The client sends a `ClientHello` with its cipher preferences and a random; the server replies with its certificate (signed up a chain to a trusted CA) and its own random, and the two parties run a key exchange (ECDHE) from which both independently derive the same symmetric session keys using the two randoms. Because the server's certificate is verified against the CA chain, the client is confident it's talking to the right host, and because ECDHE contributes fresh randomness, the session keys are new each connection (giving **forward secrecy** - a later leak of the server's private key doesn't decrypt past sessions). Only then does encrypted application data flow, using the fast symmetric keys.

The two things interviewers probe: **authentication** (the certificate chain answers "are you really `bank.com`?") and **forward secrecy** (ECDHE answers "if my private key leaks next year, is today's traffic safe?").

[Hard] Q. An attacker steals your database. What should have been true,

and what are they still missing?

Answer. Layered defence, in order of how much it helps after the breach: (1) **passwords** were stored as salted slow hashes, so the dump doesn't hand over logins - the attacker must brute-force each one; (2) **sensitive fields** (cards, SSNs, tokens) were **encrypted at rest** with keys held outside the database (KMS/HSM), so the dump is ciphertext without the key; (3) **secrets** (API keys, DB credentials) were never in the repo or the image - they live in a secrets manager; (4) access was **least-privilege** and logged, limiting what the stolen role could reach.

The answer they want is the shift in mindset: you **cannot** assume the database is safe, so you make a stolen database worth less than the cost of attacking it. The single most-cited failure is storing passwords as a plain or fast hash (or reversible-encrypted), which turns a DB breach into an instant credential breach.

[Easy] Q. What's the difference between a hash, a MAC, and a signature?

Answer.

Tool	Secret?	Proves	Who can verify
Hash (SHA-256)	none	integrity only (accidental)	anyone
MAC (HMAC)	shared key	integrity + authenticity	anyone with the key
Signature (RSA/Ed25519)	private key	integrity + authenticity + non-repudiation	anyone with the public key

A **signature** is the asymmetric upgrade of a MAC: the sender signs with a private key, and anyone can verify with the public key, and the sender can't plausibly deny it (non-repudiation). A **MAC** is symmetric - fast, but the verifier must hold the shared secret, so it can't be publicly verified. Use MACs for server<->server over a trusted channel, signatures for publicly verifiable artifacts (code, containers, documents, TLS certs).

Networking

"What happens when you type a URL into a browser?" is the single most-asked open-ended interview question. Everything here feeds into it.

[Easy] Q. TCP vs UDP?

Answer.

	TCP	UDP
Connection	Handshake first	None - just send
Delivery	Guaranteed, retransmits	Best effort, may drop
Ordering	Preserved	None
Congestion control	Yes	No
Header	20+ bytes	8 bytes
Speed	Slower	Faster, lower latency

TCP when correctness matters: HTTP, email, file transfer, databases. **UDP** when timeliness beats completeness: live video and voice, gaming, DNS.

The insight worth stating: for a video call, a packet that arrives late is useless - retransmitting it would make things worse by delaying everything behind it. Dropping it and carrying on is the correct behaviour. (QUIC, which HTTP/3 runs on, is built on UDP and reimplements reliability per-stream to avoid exactly this head-of-line blocking.)

[Medium] Q. Explain the TCP three-way handshake, and why it is three.

Answer.

- 1 **SYN** - client sends its initial sequence number.
- 2 **SYN-ACK** - server acknowledges it and sends its own.
- 3 **ACK** - client acknowledges the server's.

Three steps because **both directions** must be established, and each side must confirm the other's sequence number. Two would leave the server unsure the client received its number.

The cost: one full round trip before any data flows. On a 100 ms link that is 100 ms before the request is even sent, plus another ~2 round trips for the TLS handshake. This is why connection reuse (keep-alive), HTTP/2 multiplexing and QUIC's 0-RTT resumption all exist.

Closing takes **four** steps (FIN/ACK each way), because each direction closes independently - one side can finish sending while still receiving.

[Easy] Q. What happens when you type a URL and press enter?

Answer. The structure matters more than the detail; go layer by layer.

- 1 **URL parsing** - scheme, host, port, path.
- 2 **DNS resolution** - browser cache -> OS cache -> router -> resolver -> root -> TLD
- 3 **TCP connection** - three-way handshake to that IP on port 443.

- 4 **TLS handshake** - negotiate the cipher, validate the certificate chain
- 5 **HTTP request** - GET / HTTP/1.1 plus headers (Host, cookies, Accept).
- 6 **Server processing** - load balancer -> application server -> cache or database
- 7 **Response** - status line, headers, body.
- 8 **Rendering** - parse HTML into the DOM, fetch CSS/JS/images (more requests,

Strong candidates then mention **caching at every layer** - DNS TTL, browser cache, CDN, server-side cache - because the fastest request is the one never made.

[Medium] Q. What is the difference between HTTP/1.1, HTTP/2 and HTTP/3?

Answer.

- **HTTP/1.1** - one request at a time per connection. Browsers open ~6
 - **HTTP/2** - binary framing with **multiplexing**: many concurrent streams over
 - **HTTP/3** - runs over **QUIC on UDP**, giving each stream independent delivery.
-

[Medium] Q. What does DNS actually do, and what is a TTL?

Answer. It translates a hostname into an IP address, through a hierarchy: root servers -> TLD servers (.com) -> the domain's authoritative nameservers.

Record types worth knowing: **A** (IPv4), **AAAA** (IPv6), **CNAME** (alias), **MX** (mail), **TXT** (verification, SPF), **NS** (delegation).

TTL is how long a resolver may cache a record. It is the trade-off at the heart of DNS-based failover: a low TTL means changes propagate fast but you pay more lookups; a high TTL is efficient but leaves stale records pointing at a dead server for hours. Lowering the TTL before a planned migration is the standard practice.

[Medium] Q. What does HTTPS actually protect, and how?

Answer. TLS provides three things:

- 1 **Encryption** - an eavesdropper sees ciphertext.
- 2 **Integrity** - tampering is detected.
- 3 **Authentication** - the certificate proves you are talking to the real host.

It works by using **asymmetric** cryptography (slow) only to agree on a shared **symmetric** key (fast), then encrypting the actual traffic symmetrically. Trust comes from a chain: the site's certificate is signed by an intermediate CA, signed by a root CA that your OS or browser already trusts.

Worth knowing what it does not hide: the destination IP, and the hostname during the handshake (SNI is sent in clear unless Encrypted Client Hello is in use). An observer cannot read your traffic but can see which site you visited.

[Easy] Q. What do the HTTP status code classes mean?

Answer.

Class	Meaning	Worth knowing
1xx	Informational	101 Switching Protocols (WebSocket upgrade)
2xx	Success	200 OK, 201 Created, 204 No Content
3xx	Redirection	301 permanent, 302 temporary, 304 Not Modified
4xx	Client error	400, 401 unauthenticated, 403 unauthorised, 404, 429 rate limited
5xx	Server error	500, 502 bad gateway, 503 unavailable, 504 timeout

The distinction interviewers probe: **401 vs 403** - 401 means "I do not know who you are, authenticate"; 403 means "I know who you are and you may not do this". And **301 vs 302** - 301 is cached aggressively by browsers and is effectively permanent, so it is hard to undo.

[Medium] Q. What is the difference between a load balancer at L4 and L7?

Answer. Layer 4 balances on TCP/UDP information - IP and port. It cannot see inside the request, so it is fast and protocol-agnostic, but it can only do simple distribution.

Layer 7 parses the HTTP request, so it can route on path, host, headers or cookies (`/api/*` to one pool, `/static/*` to another), terminate TLS, retry failed requests and do sticky sessions. The cost is CPU and latency.

Use L4 for raw throughput and non-HTTP protocols; use L7 when routing decisions depend on request content, which for a typical web application is most of the time.

[Medium] Q. TCP delivers reliably over a network that drops packets. How does it

detect that a segment is lost, and how does it avoid making the loss worse?

Answer. It detects loss two ways - a **retransmission timer** expires, or it receives **three duplicate ACKs** - and it avoids making things worse by **congestion control**: every loss is treated as a signal that the network is saturated, so the sender shrinks its rate instead of just retransmitting and holding steady.

Detection, the two mechanisms:

- **Retransmission timeout (RTO).** The sender measures RTT and sets a timer
- **Fast retransmit.** If the same ACK arrives three times, the missing

Congestion control, the part most answers skip:

- The sender maintains a **congestion window** (how many unacknowledged bytes)
- On any loss signal, the window is **halved** (Reno; CUBIC, the modern Linux)
- So the answer to "why not just retransmit at full speed?" is: the loss may

The follow-up this sets up: congestion control cannot distinguish a packet lost to **congestion** from one lost to a **flaky wireless link**. On a noisy Wi-Fi connection, the flow punishes itself for the radio's mistakes, which is one reason real-time and mobile protocols either tune these parameters hard or run over UDP with their own recovery (this is a large part of why QUIC puts loss recovery in user space where it can be changed without an

OS upgrade).

Operating Systems

Asked in almost every interview that touches backend or systems work.

[Easy] Q. Process vs thread?

Answer. A **process** is a running program with its own address space. A **thread** is an execution path inside a process; threads of one process share that address space.

	Process	Thread
Memory	Isolated	Shared (heap, globals, files)
Own stack	Yes	Yes
Creation cost	High	Low
Communication	IPC (pipes, sockets, shared memory)	Shared variables
Crash blast radius	Contained	Takes down the process

The shared address space is the entire trade-off: threads communicate cheaply, and that same sharing is what makes race conditions possible. Chrome uses a process per tab precisely so one crashed tab cannot take the browser with it.

[Medium] Q. What is a context switch and why is it expensive?

Answer. Saving the CPU state of one thread (registers, program counter, stack pointer) and restoring another's, so the CPU can run something else.

The direct cost - copying registers - is small, a microsecond or so. The real cost is indirect: the new thread's data is not in the CPU caches, so it runs slowly until they refill, and a process switch also flushes the TLB because the page tables change.

This is why thread-per-request servers stop scaling around a few thousand connections, and why async I/O exists - one thread multiplexing thousands of sockets never context-switches between them.

[Medium] Q. What is virtual memory?

Answer. An abstraction giving each process its own contiguous address space, which the MMU translates to physical addresses via **page tables**.

It provides three things at once:

- **Isolation** - a process cannot name another process's memory, so it cannot
- **More memory than you have** - pages not in use can be evicted to disk and
- **Simplicity** - every program is compiled as if it owns all of memory; the

A **page fault** is the CPU trapping to the kernel because a page is not resident. A minor fault is resolved from memory (the page is already cached); a major fault requires disk I/O and is thousands of times slower. Sustained major faults are **thrashing** - the system spends all its time paging instead of working.

[Easy] Q. Stack vs heap?

Answer.

	Stack	Heap
Allocation	Move a pointer - very fast	Search a free list - slower
Lifetime	Automatic, at scope exit	Manual or garbage-collected
Size	Small and fixed (~1-8 MB)	Large
Fragmentation	None	Possible
Thread sharing	Private per thread	Shared

Local variables and call frames go on the stack; anything outliving its scope, or whose size is not known at compile time, goes on the heap. "Stack overflow" is deep recursion exhausting that small fixed region.

[Medium] Q. Explain the conditions for deadlock.

Answer. Four conditions must hold **simultaneously** (Coffman conditions); break any one and deadlock is impossible.

- 1 **Mutual exclusion** - a resource cannot be shared.
- 2 **Hold and wait** - a process holds one resource while waiting for another.
- 3 **No preemption** - resources cannot be forcibly taken back.
- 4 **Circular wait** - a cycle of processes each waiting on the next.

In practice you break **circular wait** by imposing a global lock ordering: every piece of code acquires locks in the same order, so a cycle cannot form. That is the answer to give - it is the one that is cheap and actually used.

Livelock is the related trap: threads are not blocked but keep reacting to each other and make no progress, like two people stepping aside in the same direction repeatedly.

[Medium] Q. What is the difference between concurrency and parallelism?

Answer. **Concurrency** is dealing with many things at once - a structure that lets tasks make progress in overlapping time periods. **Parallelism** is doing many things at once - literally simultaneous execution on multiple cores.

Concurrency is about program structure; parallelism is about hardware execution. A single-core machine can be concurrent (interleaving) but never parallel. An async web server is concurrent without necessarily being parallel.

The practical consequence: concurrency helps with **I/O-bound** work (waiting on network or disk), parallelism helps with **CPU-bound** work. Adding threads to a CPU-bound task on a single core makes it slower, not faster.

[Medium] Q. How does the kernel decide what runs next?

Answer. A scheduler balances throughput, latency and fairness. The approaches worth naming:

- **Round robin** - each ready thread gets a time slice in turn. Fair, simple,
- **Priority scheduling** - highest priority first; risks **starvation** of low
- **Multi-level feedback queue** - several queues; a task that uses its whole

Linux's CFS aims to give each runnable task an equal share of CPU time, tracking how much each has had and running whichever is furthest behind.

[Medium] Q. What is a system call?

Answer. The controlled entry point from user space into the kernel - the only way a program can do I/O, allocate memory or create processes.

Normal code runs in **user mode**, which cannot touch hardware or other processes' memory. A syscall traps into **kernel mode**, which can. The CPU enforces this boundary, which is what makes process isolation real rather than a convention.

Syscalls cost far more than a function call (a mode switch, argument validation, cache effects), which is why I/O is buffered - `printf` accumulates output and issues one `write` instead of one per character, and why `sendfile` exists to copy a file to a socket without crossing the boundary repeatedly.

[Medium] Q. What is a zombie process, why can it not be killed, and what is an orphan process?

Answer. A **zombie** is a process that has exited but still has an entry in the process table because its parent has not yet called `wait()` to read its exit status. It is already dead - it holds no memory, no CPU, no file descriptors - so there is nothing to kill; the entry exists only to hold the exit code until the parent collects it. An **orphan** is the mirror case: a process whose parent died first; it is alive and simply gets adopted by `init`/PID 1, which waits on it so it does not become a zombie when it exits.

Why zombies happen and what they mean:

- When a process exits, it must report **how** it exited (code or signal) to
- `kill` does not work on a zombie - there is no running code to receive the
- A single zombie is harmless (a few hundred bytes of table space). The

The practical rules you end with:

- Servers that fork workers should call `wait()` promptly, or set
- `init` (PID 1) exists partly to be the reaping backstop: every orphan is
- The classic confusion, stated out loud: `**zombie` = dead but uncollected

Virtual Memory Faults

[Medium] Q. What is the difference between a page fault and a segmentation fault?

Answer. A **page fault** is a CPU trap because a virtual page is not currently mapped as usable; it is often normal and recoverable. A **segmentation fault** is the process-visible failure after the OS determines that the access is invalid and cannot be satisfied.

On a valid page fault, the kernel may load a file-backed page, allocate a fresh zero page, or perform copy-on-write, update the page table, and restart the instruction. A major fault needs storage I/O; a minor fault can be resolved from memory.

If the address is outside mapped regions, violates permissions, or cannot be loaded, the kernel signals the process. The memorable distinction is: **page fault is the mechanism; segmentation fault is one possible outcome.**

System Design

Cache Stampede

[Hard] Q. What is a cache stampede, and how do you stop one hot key overwhelming the database?

Answer. A cache stampede happens when a popular entry expires and many requests miss together, so they all fetch or recompute the same value and send a burst to the backing store.

The protections solve different parts of the problem:

- **Request coalescing / single-flight:** one caller refreshes; others wait.
- **Jittered TTLs:** spread expirations so many keys do not fail at once.
- **Stale-while-revalidate:** serve bounded stale data during one refresh.
- **Early refresh:** probabilistically refresh hot data before expiry.

A distributed lock can coordinate hosts, but it needs a short lease and safe ownership; otherwise it becomes another outage source. Load shedding and database limits remain the final backstop because no cache strategy can make the backing service survive arbitrary demand.

System Design Fundamentals

System design interviews have no single right answer. They test whether you can reason about trade-offs out loud, ask about requirements before designing, and justify choices with numbers.

How to run the interview

- 1 **Clarify requirements** (5 min). Functional: what must it do? Non-functional:
- 2 **Estimate scale** (5 min). Get to a rough QPS and storage figure - it decides
- 3 **Sketch the high-level design** (10 min). Boxes and arrows: clients, load
- 4 **Design the data model and API** (10 min).
- 5 **Go deep** (10 min) on whatever the interviewer pushes on.
- 6 **Identify bottlenecks** (5 min) and address them.

[Easy] Q. How do you do back-of-the-envelope estimation?

Answer. Learn a handful of numbers and round aggressively.

Quantity	Approximate
Seconds in a day	~86,400, call it 10^5
1 million writes/day	~12 writes/second
Memory read	~100 ns
SSD random read	~100 μ s (1000x memory)
Disk seek (HDD)	~10 ms (100x SSD)
Same-datacentre round trip	~0.5 ms
Cross-continent round trip	~150 ms
One character (ASCII)	1 byte

Example - 100 M daily active users each posting twice:

- Writes: 200 M/day / 10^5 s ~ **2,000 writes/second**.
- Peak is typically 2-3x average: **~5,000/s**.
- At 1 KB per post: 200 GB/day, **~73 TB/year**.

That tells you immediately: one database will not hold a year of this, you need sharding, and reads will dominate so you need caching.

[Easy] Q. Vertical vs horizontal scaling?

Answer. Vertical - a bigger machine. Simple, no code changes, but there is a hard ceiling, it is expensive at the top end, and it leaves a single point of failure.

Horizontal - more machines. Effectively unbounded and fault-tolerant, but it forces you to handle statelessness, load balancing, data partitioning and distributed consistency.

The practical answer: **scale vertically until it hurts, then horizontally**. Horizontal scaling adds real complexity, and a surprising number of systems never need it. But design stateless services from the start so

the option stays open.

[Medium] Q. Where do you put caches, and how do they get stale?

Answer. Caching is the highest-leverage optimisation in most designs, because read traffic usually dominates by 10:1 or more.

Layers: browser -> CDN -> load balancer -> application (in-process) -> distributed cache (Redis/Memcached) -> database buffer pool.

Strategies:

- **Cache-aside** (most common) - app checks the cache, misses, reads the
- **Write-through** - write to cache and database together. Consistent, slower
- **Write-behind** - write to cache, flush to the database asynchronously. Fast,

Eviction: LRU is the default and usually right. (A worked LRU cache is in `examples/cpp/data-structures/lru_cache.cpp`.)

Invalidation is the hard part. TTLs are simple but serve stale data for their duration; explicit invalidation on write is precise but easy to miss a path. Watch for the **thundering herd** - a popular key expiring makes thousands of requests hit the database simultaneously. Mitigate with a lock so only one request refills, or by staggering TTLs.

[Medium] Q. Explain replication and sharding.

Answer. They solve different problems and are usually used together.

Replication - copies of the same data. Solves read scaling and availability. Primary-replica is the usual shape: writes go to the primary and propagate to replicas that serve reads. The catch is **replication lag** - a read right after a write may hit a replica that has not caught up, so the user does not see their own change. Fixes: read-your-own-writes routing to the primary, or a version token.

Sharding (partitioning) - splitting different data across machines. Solves write scaling and dataset size.

Strategy	How	Risk
Range	Keys A-M here, N-Z there	Hot spots from uneven distribution
Hash	<code>hash(key) % N</code>	Resharding moves almost everything
Consistent hashing	Keys and nodes on a ring	Adding a node moves only ~1/N of keys
Directory	A lookup service maps key -> shard	The directory is a bottleneck/SPOF

Sharding costs you cross-shard joins and transactions. Choose the shard key so the common query hits one shard.

[Hard] Q. What is CAP, and what does it actually mean in practice?

Answer. In a distributed system you can have at most two of **C**onsistency, **A**vailability and **P**artition tolerance.

The nuance most candidates miss: **partitions are not optional**. Networks fail, so P is a given. The real choice is what you do during a partition:

- **CP** - refuse requests rather than serve possibly-stale data. Banking,
- **AP** - keep serving, reconcile later. Social feeds, DNS, shopping carts -

PACELC extends it usefully: if Partitioned, choose A or C; Else, choose Latency or Consistency. That second half describes normal operation, which is where systems actually spend their time.

Eventual consistency means replicas converge if writes stop. That is weaker than it sounds, which is why stronger models exist - read-your-writes, monotonic reads, causal consistency - and naming the specific guarantee you need is a much better answer than "eventually consistent".

[Medium] Q. When do you add a message queue?

Answer. To decouple a producer from a consumer in time.

It buys you:

- **Asynchrony** - return to the user immediately, do the slow work later. (An
- **Buffering** - a traffic spike queues up instead of overwhelming the consumer.
- **Retries** - a failed message is redelivered rather than lost.
- **Fan-out** - one event, many independent consumers.

The costs are real: eventual consistency (the work is not done when you reply), message ordering is hard across partitions, and you must handle **duplicate delivery** - most queues guarantee at-least-once, not exactly-once. That makes **idempotent consumers** a requirement, not a nicety.

[Medium] Q. Rate limiting - what algorithms would you use?

Answer.

- **Fixed window** - count per calendar minute. Trivial, but allows a 2x burst
- **Sliding window log** - timestamps of every request. Exact, memory-hungry.
- **Sliding window counter** - weighted blend of the current and previous window.
- **Token bucket** - tokens refill at a steady rate; each request spends one.
- **Leaky bucket** - outflow is perfectly constant. Smooths traffic, no bursts.

In a distributed system the limiter state must be shared (typically Redis), which adds a network hop to every request - so a common design is a local limiter with a periodically synchronised global budget.

[Medium] Q. Your service must call a payment provider over the network and retry

on failure. How do you make sure a retry does not charge the customer twice?

Answer. You make the operation **idempotent** so that sending it once or five times has the same effect as sending it once. In practice that means giving the logical operation a stable **idempotency key** that you generate once (before the first attempt) and send with every attempt; the provider stores the result of the first key it has seen and returns that stored result for every repeat, instead of performing the charge again.

The reasoning, and the traps:

- **The key identifies the operation, not the attempt.** You generate it
- **Retries happen because you cannot tell "failed" from "slow".** The
- **Idempotency is a property of the receiver, triggered by the sender.** The
- **Scope the key to the thing you want to repeat.** "Charge order 1234" gets

The trade-off worth stating: idempotency keys make **at-least-once delivery safe**, which is almost always what you want over "at-most-once" (drop on failure). The cost is that the receiver keeps a deduplication record with a retention window, and the key space must be chosen so that legitimate repeat-able operations collide intentionally while distinct operations never do. This is also why HTTP distinguishes `POST` (not idempotent by default) from `PUT/DELETE` (idempotent) - a client can safely retry the latter.

Worked Example: Design an API Rate Limiter

A complete worked answer for "design the rate limiter in front of a public API," start to finish. The goal is to show the shape of a good answer: clarify, estimate, pick a data structure on purpose, design the data model, walk the request path, and name the bottlenecks you would actually hit.

1. Clarify requirements

- **What is being limited?** Requests from a client identity (API key or
- **What is the limit?** "N requests per rolling window of W seconds, per
- **What happens at the limit?** Return `429 Too Many Requests` with a
- **Where does it run?** In front of the API servers, on the hot path of
- **Multi-tenant fairness?** A single misbehaving key must not starve others,

The honest scoping line: "I'll design a **fixed-window** counter first because it is simple and good enough, then show the upgrade to **sliding window** and **token bucket** and when each is worth it."

2. Estimate the scale

- 10 M active API keys, 100 M requests/minute peak ~ **1.7 M req/s**.
- Each decision must take < ~2 ms and touch at most one shard.
- State per key is tiny (a counter, or a few counters). 10 M keys x a few

The key insight: this is a **read-mostly, per-key, low-latency** workload. The data is small and hot. That argues strongly for an **in-memory, sharded** design rather than a database on the hot path.

3. The core data structure

Three algorithms, in increasing order of fairness and cost:

- **Fixed window.** One counter per key, reset every window. `count < limit` ?
- **Sliding window log.** Store every request timestamp per key; admit if
- **Sliding window counter (the pragmatic middle).** Keep the current and
- **Token bucket** (the "smooth rate + allow bursts" variant). Each key has a

Decision to state out loud: fixed-window counter for v1 (simple, sharded, $O(1)$); token bucket if the spec allows bursts; sliding-window counter if the boundary burst is a real problem and we want to stay $O(1)$.

4. Data model

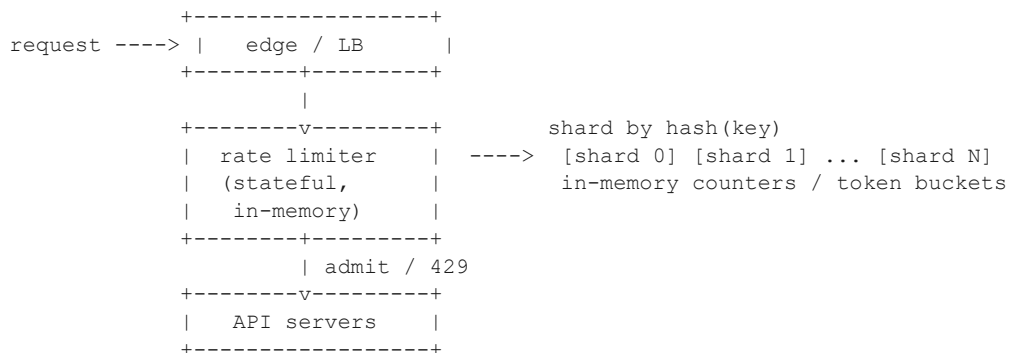
Per key, store the minimum state the chosen algorithm needs:

```
limiter_state (in memory, sharded by key)
  key          STRING    (API key / user id)  -- partition key
  window_start  TIMESTAMP
  count         INT8/INT32
  -- token bucket variant instead:
```

```
tokens          FLOAT      (current bucket level)
last_refill     TIMESTAMP
```

State is small and hot -> **keep it in memory** on the limiter nodes, sharded by hash(key). Persistence is a reliability concern, not a correctness one: if a limiter node dies we can lose that shard's counters and briefly under-count - acceptable for a limiter (failing open for a few seconds beats failing closed and rejecting everyone). Optionally replicate the hot shard to a backup node so a failure is a sub-second failover rather than a reset.

5. The architecture



- The limiter is a thin, **stateful** layer. Every request hashes its key to a
- Because state is per-shard and the decision is local, there is **no**
- 429 responses carry `Retry-After` so well-behaved clients back off.

6. The request path in detail

- 1 Edge extracts the identity (API key / auth token) and hashes it -> shard.
- 2 That shard computes the decision:
- 3 fixed window: if `now` crossed into a new window, `reset count`; then
- 4 token bucket: `refill tokens += R * (now - last_refill)` (capped at B),
- 5 Admit -> forward to the API servers. Reject -> return `429 + Retry-After`

The invariant to name: **the check-and-increment must be atomic per key**. Within one shard that is a single-threaded or lock-protected counter - no race. Across shards a key always maps to exactly one shard, so there is no cross-shard race. This is why "shard by key" is load-bearing, not just a scaling trick.

7. Bottlenecks and what you would do next

- **Hot keys**. A single abusive key hammers one shard. Mitigations: detect
- **Limiter availability**. It is now on the critical path. Run it as a
- **Clock skew**. Windows and token refill depend on `now`. Use a shared,
- **Burst at window boundary** (fixed window) -> the sliding-window-counter or
- **Global limits** ("the whole API gets 5 M req/s") need a second,

8. What the interviewer is grading

- Did you **clarify** limit semantics (per-key, rolling vs fixed, reject vs
- Did you pick the data structure **on purpose** and name the trade-off
- Did you get **sharding by key** and the **atomic per-key counter** right -
- Did you treat the limiter as a **critical component** (availability, fail
- Did you estimate scale and use it to argue for **in-memory** state?

Worked Example: Design a URL Shortener

A complete answer, start to finish, in the order you should actually deliver it. The point is the shape of the answer - the same structure works for any design question.

1. Clarify requirements

Ask before designing. Suggested questions and the answers assumed here:

Functional

- Shorten a long URL into a short one, and redirect from short to long. [yes]
- Custom aliases? Yes, optional.
- Expiry? Yes, optional TTL.
- Analytics on clicks? Basic counts, not real-time.
- User accounts, editing, deletion? Out of scope for now.

Non-functional

- Scale? 100 M new URLs per day.
- Read/write ratio? 100:1 - redirects vastly outnumber creations.
- Latency? Redirect under 100 ms at p99.
- Availability? *Highly available. A redirect failing is worse than a creation
- Consistency? A newly created link should work immediately; analytics may lag.

That last pair is the important trade: **AP for reads, with read-your-writes for the creator.**

2. Estimate the scale

```
Writes:    100 M/day / 10^5 s    ~ 1,000 URLs/second
Reads:     100 x writes          ~ 100,000 redirects/second
Peak:      ~2x average           ~ 200,000 redirects/second

Storage per record:
  short code      7 bytes
  long URL        ~500 bytes
  metadata        ~100 bytes (created_at, expiry, owner, counter)
  -----
  ~600 bytes, round to 1 KB

100 M/day x 1 KB = 100 GB/day ~ 36 TB/year
```

What these numbers tell us immediately:

- 200 K reads/second cannot come from a database -> **cache is mandatory.**
 - 36 TB/year does not fit one node -> **sharding required.**
 - Reads dominate 100:1 -> optimise the read path above everything else.
-

3. How long must the short code be?

With 62 characters (a-z A-Z 0-9):

Length	Combinations
5	$62^5 \sim 916 \text{ M}$
6	$62^6 \sim 56 \text{ B}$
7	$62^7 \sim 3.5 \text{ trillion}$

At 100 M/day we use 36.5 B/year. Six characters is exhausted in about 18 months; **seven characters gives roughly 95 years of headroom**. Choose 7.

4. API

```
POST /api/v1/urls
{ "long_url": "https://...", "custom_alias": "sale", "ttl_days": 30 }
-> 201 { "short_url": "https://sho.rt/aB3dE7x", "expires_at": "..."}

GET /{code}
-> 301 or 302 to the long URL
-> 404 if unknown or expired
```

301 vs 302 is a real decision. 301 (permanent) is cached by the browser, so subsequent visits never reach your servers - cheaper and faster, but you lose click analytics and cannot ever change the target. **302** keeps every click flowing through you. Given the analytics requirement: **302**.

5. Generating the code

Three options, and the trade-off is the answer:

(a) Hash the URL - `base62(md5(url))[0:7]`. Deterministic, so the same URL gives the same code. But collisions must be detected and resolved by rehashing with a salt, which costs a read before every write.

(b) Random generation - pick 7 random characters, retry on collision. At 3.5 trillion possibilities and billions of records, collisions are rare, but you still need a uniqueness check.

(c) Counter + base62 encoding [yes] - maintain a distributed counter and encode it. **Collision-free by construction**, no read before write, and short codes. The downsides: codes are sequential and therefore guessable and enumerable, and the counter is a coordination point.

Chosen: (c), with mitigations. Use a ticket service (or Redis `INCR`, or ZooKeeper) handing out **ranges** of 1,000 IDs to each application server, so servers only coordinate once per thousand URLs rather than per request. Then XOR/permute the counter before encoding so codes are not obviously sequential.

6. Data model

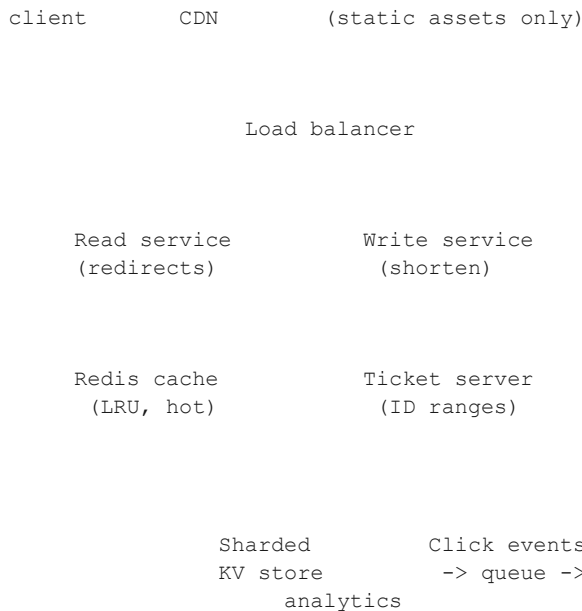
A key-value workload: look up one row by primary key, no joins, no complex queries. That points at a NoSQL store (DynamoDB, Cassandra) rather than a relational one, though a sharded relational database is entirely defensible.

```
urls
code          VARCHAR(7)  PRIMARY KEY / partition key
long_url      TEXT
created_at    TIMESTAMP
expires_at    TIMESTAMP  NULL
```

owner_id VARCHAR NULL

Shard by code using consistent hashing. Every read is by code, so every read hits exactly one shard - no scatter-gather.

7. The architecture



Read and write paths are separate services because their load differs by 100x and they should scale - and fail - independently.

8. The read path in detail

- 1 Request arrives at the read service.
- 2 Look up `code` in Redis. **Expect ~90% hit rate**: link popularity follows a
- 3 On a miss, read the shard, populate the cache, return.
- 4 Emit a click event to a queue (fire-and-forget) - **never** let analytics

Cache sizing: if 20% of URLs are hot, that is 20 M x 1 KB = **20 GB**, which fits comfortably in a small Redis cluster.

9. Bottlenecks and what you would do next

Risk	Mitigation
Cache stampede on a viral link	Single-flight refill; never expire hot keys mid-spike
Ticket server is a SPOF	Replicate it; servers hold a local range so a brief outage is invisible
Hot shard	Consistent hashing with virtual nodes
Abuse (malware, phishing)	Reputation check on creation, blocklist on redirect
Expired-record cleanup	TTL in the store, or a background sweeper

10. What the interviewer is grading

Not the diagram. They want to see that you:

- **asked about requirements before designing** - the most common failure;
- **used numbers** to justify the cache, the shard count and the code length;
- **named trade-offs explicitly** (301 vs 302, counter vs hash) rather than
- **separated read and write paths** once you knew the ratio;
- **knew what you had not solved** and could say so.

Saying "I'd choose X because of Y, though Z would be better if the requirement were W" is worth more than any particular architecture.

Language Specific

C Interview Questions

[Easy] Q. `malloc`, `calloc`, `realloc` - what does each one do, and what do they initialize?

Answer. `malloc(n)` allocates `n` bytes and returns a suitably aligned pointer, or `NULL` - the bytes are **uninitialized** (indeterminate values). `calloc(n, size)` allocates `n * size` bytes and **zero-fills** them. `realloc(p, n)` resizes the block pointed to by `p` to `n` bytes, moving it if necessary, and returns the (possibly new) pointer - `realloc(NULL, n)` is the same as `malloc(n)`, and `realloc(p, 0)` is implementation-defined.

The follow-ups:

- Check every allocation for `NULL` before using it - the size that made it
- `calloc`'s zero-fill is why it is the right choice for numeric arrays you
- The `realloc` trap that loses memory:
- The allocation is for **bytes**; `malloc(n * sizeof *p)` (not

[Easy] Q. `char *s = "abc";` **versus** `char t[] = "abc";` - what is different?

Answer. The first points **at** a string literal (read-only storage, usually in a `.rodata`-style section); the second **copies** the literal into a new array on the stack that you own. So `t[0] = 'X';` is fine, while `s[0] = 'X';` is undefined behaviour - and it is the classic "worked on the old machine, segfaults on the new one" bug, because older toolchains kept literals in writable memory.

Two more consequences interviewers probe:

- `sizeof("abc")` is 4 and `sizeof t` is 4 (both include the NUL), but
- `strlen` needs the NUL; an array like `char three[3] = {'x', 'y', 'z'};` is

[Medium] Q. Why is `sizeof(arr)` different inside the function that created the array from inside the function that received it?

Answer. Because in C an array parameter is **not** an array - the declaration `void f(int a[10])` is adjusted to `void f(int *a)`. This is called "array-to-pointer decay," and it happens at every function boundary: the callee receives a pointer, so `sizeof a` is the pointer size, and there is no length to recover. (It also means `a[3]` works in both places, because subscripting a pointer and an array is the same operation: `a[3]` is `*(a + 3)`.)

The practical consequences:

- Functions that take arrays must **also** take the length (or a NUL sentinel)
- `sizeof` is still useful on the defining side:
- Variadic `printf` solves the same information-loss problem a different way:

[Medium] Q. What are the `free()` mistakes, and why do people set pointers to NULL after freeing?

Answer. The four: **double free** (`free(p); free(p);` - the allocator's metadata is corrupt, this is UB and a classic crash/exploit primitive), **freeing what you did not allocate** (stack variables, sub-objects of a larger block, or a pointer that was advanced - `free(p + 1)` is UB), **use after free** (reading or writing through the pointer after the block went back to the pool - the memory may already belong to someone else), and **leaks** (losing the only pointer without freeing).

Setting `p = NULL` after `free(p)` does not prevent any of these by itself - it turns use after free and double free from "undefined" into "almost certainly a clean NULL dereference you can find," because `free(NULL)` is a defined no-op while `free(garbage)` is not. The real defenses are: one allocation, one free, on one pointer; short pointer lifetimes; and tools (Valgrind, ASan) that catch all four in the test run rather than in production.

[Hard] Q. What is undefined behaviour, why does the standard allow it, and which ones are the common offenders?

Answer. Undefined behaviour is any program behaviour the standard does not constrain: nothing is required to happen, including "crash," "print garbage," or "work perfectly." The standard gets away with this because compilers are allowed to **optimize under the assumption that UB never happens in the program** - code that contains UB may be transformed as if the UB-producing line could not execute, which is how "harmless" UB becomes a silent logic bug.

The common offenders, and why each is dangerous:

- **Signed integer overflow** (`INT_MAX + 1`). Because it is UB, a compiler
- **Out-of-bounds reads/writes** - including one past the end for "convenience"
- **Dangling pointers** - use after free, or a pointer to a local after return.
- **Type punning through strict aliasing** - reading an `int` through a
- **Uninitialized reads** - the value is indeterminate, and using it is UB.
- **Null dereference** - even `if (!p)` is not safe to rely on if `p`'s value

The interview-grade framing: UB is not "it probably works." It is "the compiler may assume your program is a different, well-defined program." The defenses are the same as for memory bugs - narrow pointer lifetimes, bounds carried as explicit parameters, and sanitizers/Valgrind in CI.

See also `examples/c/strings/string_literal_vs_char_array.c` and `examples/c/pointers/dynamic_int_array.c`.

C++ Interview Questions

[Easy] Q. What is RAII, and why is it the centre of C++ resource management?

Answer. Resource Acquisition Is Initialization: bind a resource (memory, file handle, lock, socket) to an object's **lifetime** - acquire it in the constructor, release it in the destructor. Because destructors run deterministically when the object goes out of scope, "cleanup" stops being a thing you remember to do and becomes a thing the language does for you, on every exit path including exceptions.

The follow-ups:

- This is why C++ has no garbage collector and no `finally`: a `std::lock_guard`
- The flip side: destructors must be **predictable and cheap-ish**, and an
- The pattern's limits: RAII owns resources with a clear owner and scope. For

[Easy] Q. `unique_ptr`, `shared_ptr`, `weak_ptr` - when is each one right?

Answer. `unique_ptr` is the default: single ownership, zero overhead (same size as a raw pointer), non-copyable but movable. `shared_ptr` is reference-counted shared ownership - use it when you genuinely cannot decide the owner up front. `weak_ptr` observes a `shared_ptr`'d object without extending its life; `lock()` upgrades to a `shared_ptr` or gives you nothing.

The trade-offs that matter in an interview:

- `shared_ptr` costs an atomic increment/decrement on every copy and a second
- **Cycles:** two `shared_ptr`s pointing at each other never hit zero and
- A `unique_ptr` can be a member; a `shared_ptr` cycle can hide for months.

[Medium] Q. What does "move semantics" actually do, and when does a move happen implicitly?

Answer. A **move** transfers a resource from one object to another instead of copying it: the moved-from object is left valid-but-unspecified (usually empty), and the expensive copy is skipped. `std::move(x)` does not move anything by itself - it is a cast to rvalue reference that permits a move constructor/assignment to be selected.

The practical rules:

- Moves happen implicitly on **prvalues** (the result of a function returning
- The classic bug: `return std::move(local);` - the `std::move` **prevents**
- A moved-from object must not be used, only assigned to or destroyed.
- The rule of zero/five: if you hand-write a destructor, copy, or move for a

[Medium] Q. `virtual` - what is the runtime cost, and what are the two traps?

Answer. A class with virtual functions gets a **vtable** (one per dynamic type, shared by all its instances) and each object carries a **vp**tr; a virtual call reads the vptr and dispatches through the table - one extra indirection, and the call usually cannot be inlined. The cost is real but small; the design cost is that the type's behaviour is no longer knowable from the static type.

The two traps (both demonstrated in `examples/cpp/oop/virtual_functions_and_the_vtable.cpp`):

- **Non-virtual destructor:** deleting a derived object through a base pointer
- **Hiding:** a non-virtual member with the same name in a derived class

The follow-up this sets up: `virtual` on the first virtual call also cannot be inlined, which is why hot single-type code sometimes uses the CRTP idiom (static dispatch through templates) to buy the call back.

[Hard] Q. Rule of zero, rule of three, rule of five - and how does a modern

codebase actually avoid all of them?

Answer. If a class owns a resource it must manage its five special members consistently: destructor, copy constructor, copy assignment, move constructor, move assignment. **Rule of three:** if you need one of the first three, you probably need all three. **Rule of five:** with move semantics, the count is five. **Rule of zero:** the best position is to need none - hold resources in types that already manage themselves (`unique_ptr`, `string`, `vector`) and let the implicit special members do the right thing.

Why this is the senior-level answer:

- Each hand-written special member is a place to get the invariant wrong -
- The modern idiom: `**own` with smart pointers and containers, inherit
- The escape hatch: when you genuinely need custom logic (a refcounted handle,

See also `examples/cpp/raii/raii_and_move.cpp` and `examples/cpp/smart-pointers/`.

C# Interview Questions

[Easy] Q. Value types vs reference types - and where does each one live?

Answer. Value types (`int`, `struct`, `enum`) are stored **in place** - on the stack when local, inline inside the object that contains them - and are copied wholesale on assignment. Reference types (`class`, `string`, `delegate`, arrays) store a reference; the object lives on the **managed heap** and assignment copies the reference, not the object.

The follow-ups that separate a definition from understanding:

- `string` is a reference type that behaves like a value type: it is
- A `struct` that holds a `string` or another class is a value type with
- The interop cost: a `struct` boxed to `object` (or put in an

See `examples/csharp/basics/value_vs_reference_types.cs`.

[Easy] Q. `==` vs `.Equals()` vs `ReferenceEquals` - when do they disagree?

Answer. `ReferenceEquals(a, b)` (and `object.ReferenceEquals`) asks "same object on the heap?" `a == b` is an operator: for most reference types it **calls** `Equals`, but `string` and types that overload `==` decide otherwise. `a.Equals(b)` is the virtual value-equality method.

The disagreements that actually bite:

- `string: "a" == "b"` compares content (the `==` operator for `string`)
 - `null: null == null` is true; `null.Equals(x)` throws
 - **Overloaded `==` without `Equals`:** a type that overloads `==` but not
 - Arrays: `==` on arrays is reference equality; element-wise comparison is
-

[Medium] Q. `async/await` - what does `await` actually do, and what is the

`async void` trap?

Answer. `await` does not block a thread. It registers the rest of the method as a continuation to run when the awaited task completes, and **returns from the method now** (the method becomes a state machine). The thread goes back to the pool; the continuation runs later, on the captured `synchronization context` (in UI code, back on the UI thread).

The traps:

- `async void` is for event handlers only. There is no `Task` to await, so
 - `.Result / .Wait()` on a task in a UI context deadlock: the thread
 - `async` **does not mean parallel**. It means asynchronous - overlapping
 - The `ConfigureAwait(false)` nuance: in library code you often don't need the
-

[Medium] Q. `interface` vs `abstract class` - and what did C# 8 default

interface members change?

Answer. An **interface** is a capability contract: no state, and (before C# 8) no implementation. A class can implement any number of them. An **abstract class** is a common base: it can carry state and shared behaviour, but a class has exactly **one** base class, so it competes with your real inheritance chain. The decision: repeated state/behaviour you want written once -> abstract base; a capability that unrelated types should share -> interface.

What C# 8 changed: interfaces can now have **default implementations**. That softens the line - an interface can carry behaviour - but it does not remove the distinction, because a default interface method still cannot touch instance state (there is none) and, more importantly, the reason to prefer an interface is the **multiple-implementation** freedom an abstract base cannot give. Default members are for extending an existing interface without breaking implementers (add a method with a sensible default), not for replacing a base class.

See `examples/csharp/oop/interface_vs_abstract_class.cs`.

[Hard] Q. What is the difference between `Task`, `ValueTask`, and

`Task<T>` - and when is each the right return type?

Answer. `Task<T>` is the standard async result: a heap-allocated object representing a computation that yields `T`. `ValueTask<T>` is the **low-allocation** alternative for methods that usually complete synchronously: it can either wrap an already-available `T` (no allocation) or a `Task<T>` (allocation happens only on the async path). `Task` (no `T`) is the fire-and-complete-notify version for work with no result.

The trade-off, stated precisely:

- **Default to** `Task<T>`. It is composable, can be stored, awaited multiple
- **Use** `ValueTask<T>` **only on hot paths** where the method frequently returns
- The interview-grade framing: `ValueTask` is an optimization with a contract

See `examples/csharp/async/` for the `Task.WhenAll` and cancellation patterns.

Go

Go interviews focus on the concurrency model (goroutines + channels), explicit error handling, and the small set of language rules that differ from C-family intuition (no inheritance, no exceptions, `interface{}` everywhere). (Snippets below validated by inspection - no Go toolchain on the authoring machine.)

[Easy] Q. What is a goroutine, and how is it different from a thread?

Answer. A goroutine is a function executed concurrently by the Go **runtime**, not the OS. The runtime multiplexes thousands of them onto a small pool of OS threads (the GMP model: Goroutines, runnables on Machine threads, driven by Ps). A goroutine starts at ~2KB of stack that grows on demand, versus ~1-8MB fixed for an OS thread - so spawning them is cheap, and the idiom is one goroutine per connection/task rather than a thread pool you size by hand.

```
package main

import "fmt"

func worker(id int, out chan<- int) {
    out <- id * id // blocks until the value is received
}

func main() {
    out := make(chan int, 3) // buffered: workers never block
    for i := 1; i <= 3; i++ {
        go worker(i, out) // start a goroutine
    }
    for i := 1; i <= 3; i++ {
        fmt.Println(<-out) // 1, 4, 9 - in some order
    }
}
```

The key mental shift: concurrency is **explicit dataflow**. Goroutines communicate by sending/receiving on channels, and a send on an unbuffered channel blocks until the other side is ready - that blocking is the synchronisation, not a lock.

[Medium] Q. Buffered vs unbuffered channels - when does each cause a goroutine to block?

Answer. An **unbuffered** channel is a direct handoff: send blocks until a receiver is waiting, and receive blocks until a sender is waiting. A **buffered** channel of size N absorbs up to N values without a waiting peer; send blocks only when the buffer is full, receive only when it is empty.

```
ch := make(chan int) // unbuffered
ch <- 1               // BLOCKS forever here: no receiver yet

bch := make(chan int, 2) // buffered, capacity 2
bch <- 1                 // fine: goes into the buffer
bch <- 2                 // fine: buffer full now
// bch <- 3             // would BLOCK: buffer full, no receiver
```

The classic deadlock is a send on an unbuffered channel (or a full buffer) with nobody receiving - the runtime kills the program when every goroutine is blocked. Rule of thumb: use unbuffered channels when you want strict handoff/synchronisation, buffered channels when you want to decouple producer and consumer rates or absorb bursts.

[Medium] Q. How does Go handle errors, and why no exceptions?

Answer. Errors are **values**: fallible functions return an extra `error` (an interface), and the caller checks it explicitly, usually immediately. There are no exceptions, so there is no hidden control flow and no "this call might throw halfway through" reasoning - every failure point is visible at the call site.

```
func divide(a, b float64) (float64, error) {
    if b == 0 {
        return 0, fmt.Errorf("division by zero")
    }
    return a / b, nil
}

func main() {
    r, err := divide(10, 2)
    if err != nil {
        fmt.Println("error:", err) // handle the failure explicitly
        return
    }
    fmt.Println(r) // 5
}
```

The standard library's `errors.Is / errors.As` and the `%w` wrap verb let you **wrap** errors to add context while keeping the original checkable: `fmt.Errorf("reading config: %w", err)` means `errors.Is(wrapped, os.ErrNotExist)` still matches the underlying `os.ErrNotExist`. The trade-off to state: verbosity (the `if err != nil` dance) in exchange for explicitness - and the idiomatic fix for deep chains is to let `?-style` helpers or a single early return at each boundary do the work.

[Medium] Q. What are interfaces in Go, and why is `io.Reader` so central?

Answer. An interface is a set of method requirements, and a type satisfies it **implicitly** by implementing the methods - no `implements` keyword. This makes interfaces small and adopted at the point of use: a function takes `io.Reader` (a single `Read([]byte) (int, error)`) and can be handed a file, a network connection, a `bytes.Buffer`, or your own type, with no changes.

```
// Anything with Read(p []byte) (int, error) is an io.Reader.
type MySource struct{ data []byte; pos int }

func (m *MySource) Read(p []byte) (int, error) {
    if m.pos >= len(m.data) {
        return 0, io.EOF
    }
    n := copy(p, m.data[m.pos:])
    m.pos += n
    return n, nil
}

// Now this generic decoder works on files, HTTP bodies, AND MySource:
func decode(r io.Reader) ([]byte, error) { return io.ReadAll(r) }
```

The design principle: **accept interfaces, return concrete types**. Small interfaces (1-3 methods) keep things **composable** - `io.Reader`, `io.Writer`, `io.Closer` combine into `io.ReadWriter`, `io.ReadCloser`, etc. The flip side to mention: implicit satisfaction can make it hard to find what implements an interface, and empty `interface{}` / `any` erases type safety until you assert.

[Hard] Q. A goroutine leaks in your service. What does that mean, how do

you find it, and how do you fix it?

Answer. A **goroutine leak** is a goroutine that can never finish - usually blocked forever on a channel send/receive, a lock, or a `select` with no ready case and no exit path. Each one holds its stack and any closures (and the data they reference), so a leak is a slow memory growth plus lost work, and it shows up as "goroutine count climbs linearly with requests" in a runtime profile.

How you find it: the runtime traceback (`/debug/pprof/goroutine?debug=1` or `go tool pprof`) groups leaked goroutines by the stack they're blocked on - you'll see hundreds parked in `<-ch` or `select`. The fixes, in order of idiom:

- 1 **Context cancellation** - thread a `context.Context` into long-running
- 2 **Ensure every channel send has a receiver** (or a buffer that will be
- 3 **Prefer `select` with an exit case** over a bare `<-ch`, and close

The answer they're looking for is the mental model: a goroutine is only as dead as its last blocking operation, so "leak" almost always means "someone stopped receiving (or stopped cancelling) and this goroutine is waiting on it forever."

Java Interview Questions

[Easy] Q. What is the difference between JDK, JRE and JVM?

Answer. **JVM** executes bytecode and is the thing that makes Java portable - one `.class` file runs anywhere a JVM exists. **JRE** is the JVM plus the standard class libraries: enough to run Java. **JDK** is the JRE plus development tools (`javac`, `jdb`, `jar`): enough to build Java.

"Write once, run anywhere" is a property of bytecode plus a per-platform JVM, not of the source language.

[Easy] Q. Primitive vs reference types?

Answer. Primitives (`int`, `double`, `boolean`, `char`, ...) hold values directly, live on the stack when local, cannot be null, and have no methods. Reference types hold a pointer to a heap object, can be null, and carry methods.

The consequence that produces bugs:

```
String a = new String("hi");
String b = new String("hi");
a == b           // false - different objects
a.equals(b)      // true  - same characters
```

`==` compares references for objects, values for primitives. **Always use `.equals()` for objects.**

Autoboxing makes this worse:

```
Integer x = 127, y = 127;
x == y;           // true  - cached (-128..127)
Integer p = 128, q = 128;
p == q;           // false - outside the cache
```

Same code, different answer depending on the value. Never compare boxed types with `==`.

[Hard] Q. Explain the equals/hashCode contract.

Answer. If `a.equals(b)` then `a.hashCode() == b.hashCode()`, **always**. The reverse is not required - equal hash codes with unequal objects is just a collision.

Override one and you must override the other. Break it and hash-based collections fail silently:

```
Map<Point, String> map = new HashMap<>();
map.put(new Point(1, 2), "origin-ish");
map.get(new Point(1, 2)); // null, if hashCode() was not overridden
```

The lookup hashes to a different bucket and never finds the entry.

The second half of the contract: **a key's hash must not change while it is in a collection**. Mutating a field used by `hashCode` strands the entry - present but unreachable, and `remove()` will not find it either. Prefer immutable keys; `record` types give you correct `equals/hashCode` for free.

[Medium] Q. Compare the collections.

Answer.

Interface	Implementation	Ordering	Notes
List	ArrayList	insertion	O(1) index, O(n) middle insert. The default.

	LinkedList	insertion	O(1) ends, poor locality - rarely the right choice
Set	HashSet	none	O(1) average
	LinkedHashSet	insertion	O(1), remembers order
	TreeSet	sorted	O(log n), needs Comparable/Comparator
Map	HashMap	none	O(1) average; chains become trees past 8 entries
	LinkedHashMap	insertion or access	access-order mode gives you an LRU cache
	TreeMap	sorted	O(log n), range queries

`ArrayList` unless you need ordering or sorting. `LinkedList` is almost never the answer despite what its Big-O table suggests - cache locality wins.

[Medium] Q. Checked vs unchecked exceptions?

Answer. **Checked** (extend `Exception`) must be declared or caught - the compiler enforces it. Intended for recoverable conditions: `IOException`, `SQLException`.

Unchecked (extend `RuntimeException`) need no declaration. Intended for programming errors: `NullPointerException`, `IllegalArgumentException`, `IndexOutOfBoundsException`.

`Error` (e.g. `OutOfMemoryError`, `StackOverflowError`) signals a JVM-level problem you should not catch.

The design debate is worth knowing: checked exceptions were meant to force handling, but in practice they encourage `catch (Exception e) {}` and leak implementation details through signatures. Later languages (C#, Kotlin, Scala) deliberately omitted them. Worked example:

`examples/java/exceptions/exception_handling_basics.java`.

[Hard] Q. What is type erasure?

Answer. Generics are a **compile-time** feature. The compiler checks types, then erases them - `List<String>` becomes plain `List` in the bytecode, with casts inserted automatically. This was done for backwards compatibility with pre-generics code.

The consequences you can be asked to explain:

```
List<String> a = new ArrayList<>();
List<Integer> b = new ArrayList<>();
a.getClass() == b.getClass();      // true - both are just ArrayList

// Illegal, because the type is not available at runtime:
// if (obj instanceof List<String>) { }
// T[] array = new T[10];
```

You also cannot overload on `List<String>` vs `List<Integer>` - after erasure both methods have identical signatures. Contrast with C#, where generics are reified and the runtime does know the type argument.

[Medium] Q. How does garbage collection work?

Answer. The JVM reclaims objects no longer **reachable** from GC roots (stack references, statics, JNI references). Note it is reachability, not reference counting - so cycles are collected correctly.

The design rests on the **generational hypothesis**: most objects die young. So the heap is split:

- **Young generation** (Eden + two survivor spaces) - collected frequently with
- **Old generation** - objects that survived several collections; collected

Modern collectors (G1, ZGC, Shenandoah) do most work concurrently to keep pause times low - ZGC targets sub-millisecond pauses on large heaps.

`System.gc()` is a suggestion the JVM may ignore. You cannot force collection, and calling it is almost always a mistake. Memory leaks in Java are real but take a different form: objects still referenced but never used - a static collection that only ever grows, an unremoved listener, an unclosed resource.

[Medium] Q. What does the Stream API give you?

Answer. A declarative pipeline over a sequence, with lazy intermediate operations and one terminal operation that triggers execution.

```
List<String> names = people.stream()
    .filter(p -> p.age() >= 18)           // intermediate, lazy
    .map(Person::name)                   // intermediate, lazy
    .sorted()
    .toList();                          // terminal, runs the pipeline
```

Nothing executes until the terminal operation, which allows fusion and short-circuiting (`findFirst` stops early).

Two cautions worth raising unprompted: a stream is **single-use** - reusing one throws

`IllegalStateException`; and `parallelStream()` is not free speed. It uses the common `ForkJoinPool`, so it helps only for large datasets with CPU-bound, independent, side-effect-free work, and can be slower otherwise. Worked example: `examples/java/streams/group_and_collect_with_streams.java`.

[Medium] Q. `String`, `StringBuilder`, `StringBuffer`?

Answer. `String` is **immutable** - every concatenation allocates a new object, so building in a loop is $O(n^2)$.

`StringBuilder` is a mutable buffer, not thread-safe, and is what you want. `StringBuffer` is the synchronised version: slower, and almost never needed since a builder is normally local to one thread.

Note the compiler rewrites simple + concatenation into builder calls, so `"a" + b + "c"` is fine - it is concatenation **inside a loop** that is the problem. Worked example:

`examples/java/strings/string_immutability_and_stringbuilder.java`.

JavaScript Interview Questions

[Easy] Q. var, let and const?

Answer.

	Scope	Hoisting	Reassign
var	function	hoisted, initialised undefined	yes
let	block	hoisted into the temporal dead zone	yes
const	block	same as let	no

const prevents **rebinding the name**, not mutation of the value:

```
const user = { name: "Sara" };
user.name = "Ali";           // fine - the object is mutable
user = {};                   // TypeError - cannot rebound
```

Use `const` by default, `let` when you must reassign, and `var` never.

[Medium] Q. What is a closure?

Answer. A function that retains access to the variables of the scope it was created in, even after that scope has returned.

```
function counter() {
  let count = 0;                // stays alive via the closure
  return {
    increment: () => ++count,
    value: () => count,
  };
}
const c = counter();
c.increment(); c.increment();
c.value();      // 2
```

`count` is genuinely private - unreachable except through the returned functions. This is the basis of the module pattern, memoisation, and every callback that "remembers" something.

The classic interview trap:

```
for (var i = 0; i < 3; i++) setTimeout(() => console.log(i)); // 3, 3, 3
for (let i = 0; i < 3; i++) setTimeout(() => console.log(i)); // 0, 1, 2
```

`var` is function-scoped, so all three callbacks close over **one** variable, which is 3 by the time they run. `let` creates a fresh binding per iteration. Worked example: [examples/javascript/basics/closures.js](https://github.com/InterviewCracker/javascript-examples/blob/master/javascript/basics/closures.js).

[Hard] Q. Explain the event loop.

Answer. JavaScript is single-threaded but non-blocking. The runtime has:

- a **call stack** - what is executing now;
- **Web APIs / host APIs** - timers, network, file I/O, running outside the
- a **macrotask queue** - `setTimeout`, I/O callbacks, UI events;
- a **microtask queue** - promise callbacks, `queueMicrotask`.

The loop: when the stack empties, **drain the entire microtask queue**, then take **one** macrotask, then drain microtasks again. Microtasks always have priority.

```
console.log("1");
setTimeout(() => console.log("2"), 0); // macrotask
Promise.resolve().then(() => console.log("3")); // microtask
console.log("4");
// 1, 4, 3, 2
```

2 comes last despite a 0 ms delay, because the microtask queue is drained first. The follow-up: an infinite chain of microtasks **starves** the macrotask queue and freezes the page - microtask priority is not free. Worked example: `examples/javascript/async/promises_and_event_loop.js`.

[Medium] Q. How does `this` get its value?

Answer. In a normal function `this` is determined by **how it is called**, not where it is defined. Four rules, in precedence order:

- 1 **new** - `this` is the new object.
- 2 **Explicit** - `call`, `apply`, `bind` set it.
- 3 **Implicit** - `obj.method()` sets `this` to `obj`.
- 4 **Default** - `undefined` in strict mode, `globalThis` otherwise.

Arrow functions ignore all four: they have **no** `this` **of their own** and inherit it lexically from the enclosing scope. That is the fix for the classic bug:

```
class Timer {
  constructor() { this.seconds = 0; }
  startBroken() { setInterval(function () { this.seconds++; }, 1000); } // wrong `this`
  start()       { setInterval(() => { this.seconds++; }, 1000); }      // correct
}
```

Worked example: `examples/javascript/basics/this_and_binding.js`.

[Medium] Q. `==` vs `===`?

Answer. `===` compares type and value with no conversion. `==` coerces first, using rules that are genuinely surprising:

```
0 == ""; // true
0 == "0"; // true
"" == "0"; // false <- not transitive!
null == undefined // true
null == 0 // false
NaN == NaN // false
```

Always use `===`. The one accepted exception is `x == null`, which checks for `null` or `undefined` in a single test.

For `NaN`, use `Number.isNaN(x)`. `NaN !== NaN` is required by IEEE 754 and is also why `[NaN].includes(NaN)` is true (it uses `SameValueZero`) while `[NaN].indexOf(NaN)` is `-1` (it uses `===`).

[Medium] Q. What is prototypal inheritance?

Answer. Every object has a hidden link to another object, its **prototype**. A property miss walks up that chain until it is found or the chain ends at `null`.

```
const animal = { speak() { return `${this.name} makes a sound`; } };
const dog = Object.create(animal);
dog.name = "Rex";
dog.speak(); // found on the prototype
```

`class` syntax is sugar over this - there are no classical classes underneath, just constructor functions and prototype objects. That is why you can add a method to every existing instance at runtime by assigning to the prototype, which classical inheritance does not allow.

[Medium] Q. Promise, async/await, and how do you run things in parallel?

Answer. A promise represents a value that is not ready yet, in one of three states: pending, fulfilled, rejected. `async/await` is syntax over promises - an `async` function always returns one, and `await` suspends until it settles.

The mistake that matters:

```
// Sequential - 3 seconds, and usually a bug
const a = await fetchA();
const b = await fetchB();

// Parallel - 1 second
const [a, b] = await Promise.all([fetchA(), fetchB()]);
```

If the calls are independent, awaiting them one at a time wastes the whole latency of each. Know the four combinators:

- `Promise.all` - all succeed, or reject on the **first** failure.
- `Promise.allSettled` - wait for all, never rejects; inspect each result.
- `Promise.race` - first to **settle**, success or failure (used for timeouts).
- `Promise.any` - first to **succeed**.

Worked example: `examples/typescript/async/async_await_parallel_vs_sequential.ts`.

[Medium] Q. Debounce vs throttle?

Answer. Both limit how often a function runs, in opposite ways.

- **Debounce** - wait until the events stop, then run once. Search-as-you-type,
- **Throttle** - run at most once per interval while events continue. Scroll

"Fire after the user stops typing" is debounce; "fire at most every 200 ms while scrolling" is throttle. Worked example: `examples/javascript/utilities/debounce_with_cancellation.js`.

Python Interview Questions

[Easy] Q. Mutable vs immutable - and why does it matter?

Answer. Immutable: int, float, str, tuple, frozenset, bytes. Mutable: list, dict, set, bytearray, most custom classes.

It decides three things: whether a value can be a dict key (hashable, which requires immutable), whether passing it to a function can change the caller's object, and whether it is safe to share between threads.

The classic bug it causes:

```
def append_to(item, target=[]):      # DANGER: evaluated once, at def time
    target.append(item)
    return target

append_to(1)    # [1]
append_to(2)    # [1, 2] <- the same list, still there
```

Default arguments are evaluated **once**, when the function is defined. Use `None` as the sentinel:

```
def append_to(item, target=None):
    if target is None:
        target = []
    target.append(item)
    return target
```

[Easy] Q. is vs ==?

Answer. `==` compares **values** (via `__eq__`); `is` compares **identity** - whether both names point to the same object.

```
a = [1, 2, 3]
b = [1, 2, 3]
a == b    # True - same contents
a is b    # False - two distinct objects
```

Use `is` only for singletons: `is None`, `is True`, `is False`.

The confusing part is small-integer and string caching. `256 is 256` is `True` because CPython interns small integers (-5 to 256); `1000 is 1000` may be `False` depending on context. This is an implementation detail you must never rely on - and a favourite trick question.

[Hard] Q. What is the GIL and what does it mean for you?

Answer. The Global Interpreter Lock is a mutex ensuring only one thread executes Python bytecode at a time, even on a many-core machine. It exists because CPython's reference-counting memory management is not thread-safe, and one coarse lock was far simpler and faster for single-threaded code than locking every object.

The practical consequence:

- **CPU-bound work gets no speedup from threads.** Four threads on four cores
- **I/O-bound work does benefit**, because the GIL is released around blocking

Worth knowing this is changing: PEP 703 adds an optional free-threaded build (3.13+) with no GIL, and 3.12+ supports per-interpreter GILs. Saying "the GIL is being removed as an option, gradually" is more accurate than either "it's gone" or "Python can't do parallelism".

[Medium] Q. What is a generator and why use one?

Answer. A function containing `yield`. Calling it returns a lazy iterator that computes values on demand rather than building a list.

```
def read_large_file(path):
    with open(path) as f:
        for line in f:
            yield line.strip()          # one line in memory, not the whole file
```

Two benefits: **constant memory** regardless of input size, and the ability to represent infinite sequences. The costs: you can only iterate once, and you cannot index or take `len()`.

Use `()` instead of `[]` to get a generator expression: `sum(x*x for x in range(10**8))` never allocates the list. Worked example: `examples/python/generators/log_pipeline_with_generators.py`.

[Medium] Q. What is a decorator?

Answer. A callable that takes a function and returns a replacement, used to add behaviour without editing the original.

```
import functools, time

def timed(func):
    @functools.wraps(func)          # preserve __name__, __doc__, signature
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        try:
            return func(*args, **kwargs)
        finally:
            print(f"{func.__name__} took {time.perf_counter() - start:.4f}s")
    return wrapper

@timed
def work(): ...

@timed is exactly work = timed(work).
```

`functools.wraps` **is the detail interviewers check for**. Without it the decorated function reports the wrapper's name and loses its docstring, which breaks introspection, documentation tools and debugging. Worked example: `examples/python/decorators/retry_decorator_with_backoff.py`.

[Medium] Q. Shallow vs deep copy?

Answer. A **shallow copy** (`list(x)`, `x[:]`, `copy.copy`) makes a new outer object holding **references to the same inner objects**. A **deep copy** (`copy.deepcopy`) recursively copies everything.

```
import copy
original = [[1, 2], [3, 4]]
shallow = copy.copy(original)
shallow[0].append(99)
original          # [[1, 2, 99], [3, 4]]  <- mutated!

deep = copy.deepcopy(original)
deep[0].append(100)
original          # unchanged
```

Deep copy is slower and fails or loops on some structures - it handles cycles, but not open file handles or sockets. Prefer immutable data over copying.

[Medium] Q. What does a context manager do?

Answer. Guarantees cleanup via `__enter__` / `__exit__`, so a resource is released even if an exception is raised.

```
from contextlib import contextmanager

@contextmanager
def timer(label):
    start = time.perf_counter()
    try:
        yield
    finally:
        # runs on exception too
        print(f"{label}: {time.perf_counter() - start:.3f}s")
```

`with open(...)` is the everyday example: the file closes on any exit path. Worked example: `examples/python/context-managers/custom_context_managers.py`.

[Medium] Q. `__str__` vs `__repr__`?

Answer. `__repr__` is for **developers** - unambiguous, ideally valid Python that would recreate the object. It is what the REPL and containers show. `__str__` is for **users** - readable.

If you implement only one, implement `__repr__`: `__str__` falls back to it, but not the other way round. Worked example: `examples/python/oop/dunder_methods.py`.

[Medium] Q. What are `*args` and `**kwargs`?

Answer. `*args` collects extra positional arguments into a tuple; `**kwargs` collects extra keyword arguments into a dict. In a call, the same symbols unpack:

```
def f(a, b, c): ...
values = [1, 2, 3]
f(*values)                # unpacks positionally

config = {"a": 1, "b": 2, "c": 3}
f(**config)               # unpacks by keyword
```

This is what lets a decorator wrap any function signature.

Rust

Rust interviews test one idea above all: do you understand **ownership**, and can you reason about when the borrow checker accepts or rejects code? The language features (traits, enums, iterators) are the icing; memory safety without a garbage collector is the substance.

[Easy] Q. What is ownership, and what does it mean to "move" a value?

Answer. Every value has exactly one **owner**; when the owner goes out of scope, the value is dropped. Assignment and function arguments **move** the value by default - the original binding is no longer usable. This is what guarantees no double-free and no use-after-free without a GC.

```
fn takes_ownership(x: String) -> u32 {
    x.len() as u32
}

fn main() {
    let s = String::from("hello");
    let n = takes_ownership(s);    // `s` is MOVED into the function
    assert_eq!(n, 5);
    // println!("{}", s);          // COMPILER ERROR: `s` was moved
}
```

Types that implement `Copy` (`ints`, `bool`, `floats`, `char`, and `fixed-size arrays of those`) are copied instead of moved, which is why `let y = x;` feels free for `i32` but not for `String`.

[Medium] Q. What are borrowing and lifetimes, and when does the borrow

checker reject your code?

Answer. A **borrow** (`&T` / `&mut T`) lets you use a value without taking ownership. The rules: any number of shared borrows, OR exactly one mutable borrow - never both at once. A **lifetime** annotation tells the compiler how long a reference may outlive the data it points to; you rarely write them explicitly, but `longest<'a>(a: &'a str, b: &'a str) -> &'a str` shows the rule that the returned reference may live no longer than the shorter-lived input.

```
fn longest<'a>(a: &'a str, b: &'a str) -> &'a str {
    if a.len() >= b.len() { a } else { b }
}

fn first_word(s: &str) -> &str {
    s.split_whitespace().next().unwrap_or("")
}

fn main() {
    let s1 = String::from("long string is long");
    let s2 = String::from("xyz");
    let result = longest(s1.as_str(), s2.as_str());
    assert_eq!(result, "long string is long");
    assert_eq!(first_word(" hello world"), "hello");
}
```

The classic rejection, and the one to be able to explain out loud:

```
fn main() {
    let result;
    {
        let s = String::from("hello");
        result = longest(s.as_str(), "world");
    }                                // `s` dropped here
    let _ = result.len();           // ERROR: `s` does not live long enough
}
```

```
}
```

`result` borrows from `s`, which dies at the closing brace - returning a reference to it would dangle. The fix is to return an owned `String` (`.to_string()`) or keep `s` alive for as long as `result` is used.

[Medium] Q. How does Rust handle errors - `Option`, `Result`, and the `?`

operator?

Answer. Failure is a **value**, not an exception. `Option<T>` is `Some(T)` or `None` for "no value"; `Result<T, E>` is `Ok(T)` or `Err(E)` for recoverable failures. The `?` operator propagates an error: on `Err` it returns early from the current function, on `Ok` it unwraps the value - replacing the nested-matching pyramid of code.

```
fn parse_int(s: &str) -> Result<i64, String> {
    s.parse::<i64>().map_err(|e| format!("not a number: {e}"))
}

fn safe_div(a: i64, b: i64) -> Result<i64, String> {
    if b == 0 { return Err("division by zero".to_string()); }
    Ok(a / b)
}

// `?` returns early on Err, unwraps on Ok:
fn process(input: &str) -> Result<i64, String> {
    let n = parse_int(input)?;
    let half = safe_div(n, 2)?;
    Ok(half)
}

fn main() {
    assert_eq!(process("10"), Ok(5));
    assert!(process("abc").is_err());
    assert_eq!(safe_div(1, 0), Err("division by zero".to_string()));
}
```

The trade-off to state: `?` makes pipelines linear but forces every function to thread a `Result` through its signature - errors are explicit at every boundary (a feature), whereas exception-based languages let failures surface only where they are caught (and silently, if never caught).

[Medium] Q. When do you reach for `Box`, `Rc`, or `RefCell`?

Answer. They solve the three limitations of the stack-ownership model:

- `Box<T>` - heap allocation / unknown-at-compile-time size /
- `Rc<T>` - shared ownership: many owners, one data. Reference
- `RefCell<T>` - interior mutability: immutable binding, mutable

```
use std::rc::Rc;
use std::cell::RefCell;

struct Node {
    value: i32,
    next: RefCell<Option<Rc<Node>>>,
}

fn push(head: &Rc<RefCell<Option<Rc<Node>>>>, value: i32) {
    let new = Rc::new(Node { value, next: RefCell::new(None) });
    (*head).borrow_mut().replace(new); // unique write access for a moment
}

fn len(head: &Rc<RefCell<Option<Rc<Node>>>>) -> usize {
```

```

    let mut n = 0;
    let mut cur = head.borrow().clone(); // clone the Rc (cheap refcount bump)
    while let Some(node) = cur {
        n += 1;
        cur = node.next.borrow().clone();
    }
    n
}

```

`Rc` outside + `RefCell` inside is the standard shape for data with multiple owners AND runtime-rearranged structure (graphs, trees you mutate in place). The cost: `borrow()/borrow_mut()` panic at runtime on rule violations, and `Rc` increments/decrements a counter on every clone/drop.

[Hard] Q. Walk through what happens when a function with an `Option`

return value is called, and why `unwrap()` is a code smell in production.

Answer. The caller **must** handle both cases - the type system makes `None` a visible branch, usually via `match`, `if let`, or `? .unwrap()` (and `expect()`) say "I guarantee this is `Some`, panic otherwise" - fine in tests and prototypes, but in production it converts a handleable absence into a crash, typically deep in a request path where the real cause (a bad input, a missing config key) is far away. The production pattern is to propagate: return a `Result/Option` upward with `?`, map the `None` into a meaningful error with `ok_or(...)`, or branch deliberately with a default only when a default is genuinely correct.

```

fn find_user(id: u64) -> Option<String> {
    (id == 1).then(|| String::from("ada"))
}

fn greet(id: u64) -> Result<String, String> {
    let user = find_user(id)
    .ok_or_else(|| format!("no user with id {id}"))?;
    Ok(format!("hello {user}"))
}

fn main() {
    assert_eq!(greet(1), Ok("hello ada".to_string()));
    assert!(greet(2).is_err());
}

```

The follow-up: `Option` and `Result` are enums with **zero overhead** in the `Some/Ok` case (a niche optimization means `Option<&T>` is the same size as `&T`), so wrapping fallible values costs nothing at runtime - the cost is purely at the call sites, which is exactly where you want the attention.

Behavioural

Behavioural Interviews

Technically strong candidates lose offers here more often than they expect, usually for one reason: they answer in generalities instead of telling a specific story.

The STAR structure

		Time
Situation	The context. Where, when, what was at stake.	~15%
Task	Your specific responsibility.	~15%
Action	What you did, and why you chose it.	~55%
Result	The outcome, quantified where possible, and what you learned.	~15%

Two rules that fix most weak answers:

- 1 **Say "I", not "we"**. The interviewer is assessing you, not your team. "We
 - 2 **Spend the time in Action**. Candidates over-explain the setup and then
-

Preparing properly

Do not prepare an answer per question - there are too many. Prepare **five to seven stories** and map them onto question types. Most behavioural questions are one of these:

Theme	What they are checking
Conflict with a colleague	Can you disagree without damaging a relationship
A project that failed	Do you take ownership, and did you learn
Leadership / influence	Can you move things without authority
A difficult technical decision	How you reason under uncertainty and trade-offs
Handling pressure or ambiguity	Do you prioritise, or freeze
Feedback you received	Are you coachable
Something you are proud of	What you actually value

A good story is **specific, recent, yours, and has a measurable outcome**. Write each one out once, in full. You will not memorise it - but writing it makes the details available under pressure.

The questions that recur, and what a good answer looks like

"Tell me about yourself."

Not your life story. Three parts, about 90 seconds: where you are now and what you own, one or two accomplishments relevant to this role, and why this role is the next logical step. Finish with a clear handover rather than trailing off.

"Tell me about a time you disagreed with a colleague."

They are testing whether you can hold a position without making it personal. A strong answer shows you sought to understand the other view first, brought **data** rather than opinion, and - crucially - either changed your mind or committed fully to the decision that was made. "I was right and they eventually agreed" is a weaker answer than it feels.

Avoid: stories where the other person is simply incompetent. It reads as a red flag about you, not them.

"Tell me about a project that failed."

The trap is choosing something trivial, or something that was not your fault. Pick a real failure with genuine consequences, take clear ownership of your part, and spend most of the answer on what you changed afterwards - and ideally, on evidence that the change stuck.

Avoid: "my weakness is that I work too hard." It signals that you either lack self-awareness or are unwilling to be honest.

"Tell me about a difficult technical decision."

This is really a system-design question in narrative form. Name the options you considered, the constraints (deadline, team skills, existing systems, cost), why you chose as you did, and what you would do differently knowing what you know now. Acknowledging a trade-off you accepted is a strength, not an admission.

"Why do you want to work here?"

Do the research. Reference something specific - the product, a technical blog post, the problem domain - and connect it to what you want to do next. Generic praise is transparently generic.

"Do you have any questions for us?"

Always yes. This is assessed. Useful ones:

- What does success look like for this role in the first six months?
- How does the team decide what to work on?
- What is the on-call and code review culture like?
- What is the biggest technical challenge the team faces right now?
- Why is this role open?

Common mistakes

- **Rambling.** Aim for two to three minutes. Practise out loud and time
- **Hypotheticals.** "I would usually..." is not a story. Give a specific instance.
- **No result.** Every story needs an ending, with a number if one exists
- **Blaming.** Even when someone else genuinely caused the problem, focus on your

- **No preparation.** It shows immediately, and interviewers read it as not
-

A note on honesty

Do not invent stories. Interviewers ask follow-up questions - "what would you do differently?", "how did the other person react?", "what was the number before?" - and fabricated stories collapse under two or three of them. A modest true story, told well and with real reflection, beats an impressive invented one every time.

If you genuinely lack experience with something, say so and describe the nearest thing you have done. "I have not led a team, but I did own the migration end-to-end and coordinated three people across two teams" is a good answer. A fabricated one is a failed reference check waiting to happen.